Tabular Data Wrangling in the Era of Large Language Model: A Survey

Ying He, Qiang Yang, Yuxiao Yang, Yubo Zhou, Zhecheng Hu, Sheng Yuan, Yanghua Xiao, and Zhixu Li

(Survey Paper)

***Abstract*—Data quality is a critical factor in data-centric artificial intelligence (AI), as "dirty data" can significantly hinder analysis and degrade downstream model performance. Among various data formats, tabular data is a ubiquitous and essential form of structured information, widely used across domains such as business, finance, healthcare, and scientific research. Tabular data wrangling, encompassing tasks, such as error correction, missing value imputation, data normalization, semantic interpretation, and the integration of heterogeneous sources, is a foundational step in the data preparation pipeline. While traditional statistical and deep learning approaches have shown competitive results, they often face limitations in robustness, manual effort, and generalization. Recent advances in large language models (LLMs), whose capabilities have expanded from unstructured text to structured data through techniques like prompt engineering and supervised fine-tuning, offer promising alternatives for addressing these challenges. Although prior surveys have examined LLM applications in downstream tabular tasks, such as question answering and fact verification, limited attention has been given to their potential in improving data quality. This survey aims to fill that gap by providing a comprehensive overview along four dimensions: 1) formal definitions and categorization of tabular wrangling tasks, 2) the typical workflow of applying LLMs to tabular data, 3) the roles LLMs can play in various wrangling methods, and 4) a comparative analysis of LLM-based versus traditional approaches, highlighting their respective strengths and limitations.**

Received 15 May 2025; revised 15 April 2026; accepted 20 July 2026. Date of publication 24 July 2026; date of current version 15 September 2026. This work was supported in part by the Key-Area Research and Development Program of Guangdong Province under Grant 2024B0101050005, in part by the Natural Science Foundation of Jiangsu Province under Grant BK20251823, in part by the Beijing Natural Science Foundation under Grant 4262049, in part by the Suzhou Key Laboratory of Artificial Intelligence and Social Governance Technologies under Grant SZS2023007, in part by the Smart Social Governance Technology and Innovative Application Platform under Grant YZCXPT2023101, in part by the Innovation System of the Integration between Industry and Education for Smart Governance under Grant CJRH2024101, and in part by the Leadership Talent Program (Science and Education) of SIP. Recommended for acceptance by Y. Shen. *(Corresponding authors: Yanghua Xiao; Zhixu Li.)*

Ying He, Yubo Zhou, Zhecheng Hu, and Yanghua Xiao are with the College of Computer Science and Artificial Intelligence, Fudan University, Shanghai 200437, China (e-mail: yinghe23@m.fudan.edu.cn; zhouyb24@m.fudan.edu.cn; zchu24@m.fudan.edu.cn; shawyh@fudan.edu.cn).

Qiang Yang is with the College of Medicine, University of Florida, Gainesville, FL 32611 USA (e-mail: qiangyang@ufl.edu).

Yuxiao Yang and Sheng Yuan are with the School of Smart Governance, Renmin University of China, Suzhou 215000, China (e-mail: yangyuxiao@ruc.edu.cn; yuanshengyes@ruc.edu.cn).

Zhixu Li is with the School of Information and the School of Smart Governance, Renmin University of China, Beijing 100086, China (e-mail: zhixuli@ruc.edu.cn).

Digital Object Identifier 10.1109/TKDE.2026.3716977

***Index Terms*—Tabular data, data wrangling, large language model.**

I. Introduction

DATA wrangling, a crucial process in Data-centric Artificial Intelligence (AI), involves transforming raw, inconsistent, and messy data into structured and usable formats. Studies [1], [2], [3] have shown that data scientists spend between 50% and 80% of their time on data wrangling, largely due to the prevalence of "dirty data". Among various data formats, tabular data represents over 73% of the total [4], owing to its rich semantic content that includes textual descriptions, numerical data, visual formatting, and embedded formulas. Given its complexity and ubiquity, this survey focuses specifically on wrangling techniques for tabular data, encompassing correction, imputation, normalization, integration, and interpretation tasks, as illustrated in Fig. 1(a)–(e).

Traditional tabular data wrangling approaches rely heavily on heuristic rules [5], [6], [7], manual interventions [8], [9], and handcrafted features [6], [10]. While effective in constrained settings, these methods often fall short in real-world applications with large volumes of dynamic and heterogeneous data. Recent advances in deep learning have introduced the *pre-training and fine-tuning* paradigm, which has been extended to tabular data wrangling tasks with promising results [11], [12], [13], [14]. However, due to the limited number of model parameters and insufficiency of pre-training corpora, several key challenges persist: (1) the need for substantial task-specific labeled data for effective fine-tuning; (2) limited capacity to capture complex contextual dependencies; and (3) significant performance degradation on unseen datasets or novel tasks.

Large Language Models (LLMs), such as GPT series [15], [16], LLaMA [17], PaLM [18], and Qwen [19], with billions of parameters, have exhibited remarkable performance across a wide range of natural language processing (NLP) tasks, including reading comprehension and multi-step reasoning [20]. Their strong contextual understanding and generative capabilities present opportunities for overcoming the limitations of traditional and earlier deep-learning methods (collectively referred to as *LLM-free methods*), such as reliance on extensive labeled data, and poor generalization [21]. Notably, LLMs also show potential in handling non-textual data (e.g., tabular data, code

1041-4347 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

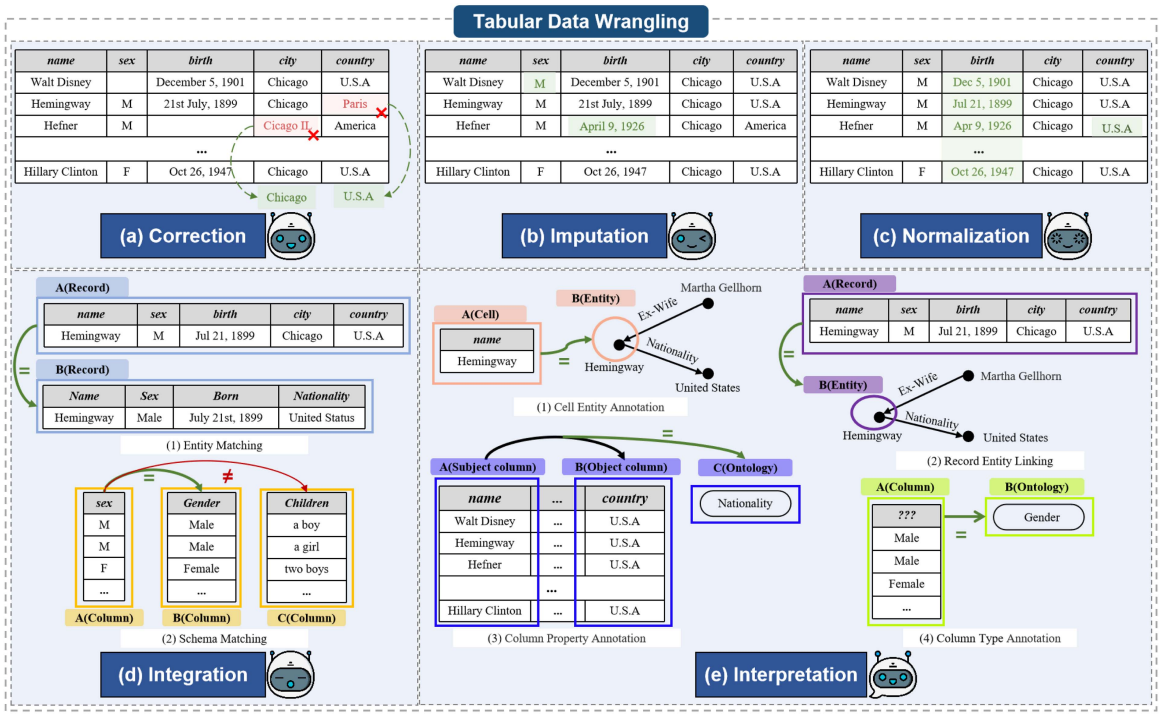


Fig. 1. Five core tasks in the tabular data wrangling process depending on the types of "dirty data".

TABLE I
COMPARISON WITH OTHER LLM-FOR-TABLE SURVEYS

Paper	Year	Table-related	TC	TIp	TN	TInt	TItp
[22]	2025	✓	✓	✗	✗	✗	✓
[23]	2024	✓	✓	✗	✗	✗	✗
[24]	2024	✓	✗	✗	✗	✗	✓
[25]	2025	✓	✓	✓	✗	✗	✓
[26]	2024	✓	✗	✓	✗	✗	✓
[27]	2025	✓	✗	✗	✗	✗	✗
[28]	2025	✓	✓	✓	✗	✗	✗
[29]	2025	✓	✗	✗	✗	✗	✗
Ours	2025	✓	✓	✓	✓	✓	✓

and mathematical formulas), sparking growing interest in their application to tabular data wrangling.

However, to the best of our knowledge, no existing survey provides a comprehensive overview of LLM-based methods for tabular data wrangling tasks, as summarized in Table I. To fill this gap, we conducted a systematic literature search using a set of representative keywords (e.g., table cleaning, schema matching, record linkage), as well as related extensions (e.g., table reasoning, table understanding). Finally, a unified taxonomy is summarized in Fig. 2.

Contributions: The notable contributions of this survey are summarized as follows:

- *Method Taxonomy:* We propose a fine-grained taxonomy of tabular data wrangling techniques, with a focus on developments in the era of Large Language Models.
- *Comprehensive Review:* We systematically survey recent LLM-based methods for five tabular data wrangling tasks.
- *Critical Comparison:* We provide a detailed comparison between LLM-based and LLM-free methods, highlighting their respective strengths and limitations.
- *Future Directions:* We outline open challenges and suggest future research directions to advance LLM-based tabular data wrangling.

II. BACKGROUND

In this section, we first define essential terminologies and core tasks in tabular data wrangling, then review LLM-free methods and their limitations. We also briefly discuss representative work on table understanding and reasoning, which offers important foundations for LLMs to understand tabular structures and reason over tables. For clarity, we use "table" to refer to "tabular data" throughout the rest of this paper.

A. Definitions

1) Data Elements: In this survey, a *data element* refers to an atomic unit in a table, such as a cell, a record (row), a column, or other structural components.

Definition 1. (Table): A table T is a structured arrangement of data organized into rows and columns. Formally, it is defined as a two-dimensional structure $T = (A, V)$, where $a_i \in A$ represents the i-th attribute (also referred to as a header or column name), and $v_{ji} \in V$ denotes the value corresponding to a_i in the j-th row.

Definition 2. (Cell): A cell c_{ji} is the atomic entry located at the intersection of the j-th row and the i-th column in table T. Cells can be categorized as either header cells or data cells. A header cell defines an attribute and provides semantic context for its column, while a data cell contains a value corresponding to the attribute for a specified row.

Definition 3. (Column): A column C_i contains all values in the i-th column of table T, represented as $\{v_{ji}\}_{1 \le j \le m}$, where m is the number of records (rows) in T.

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

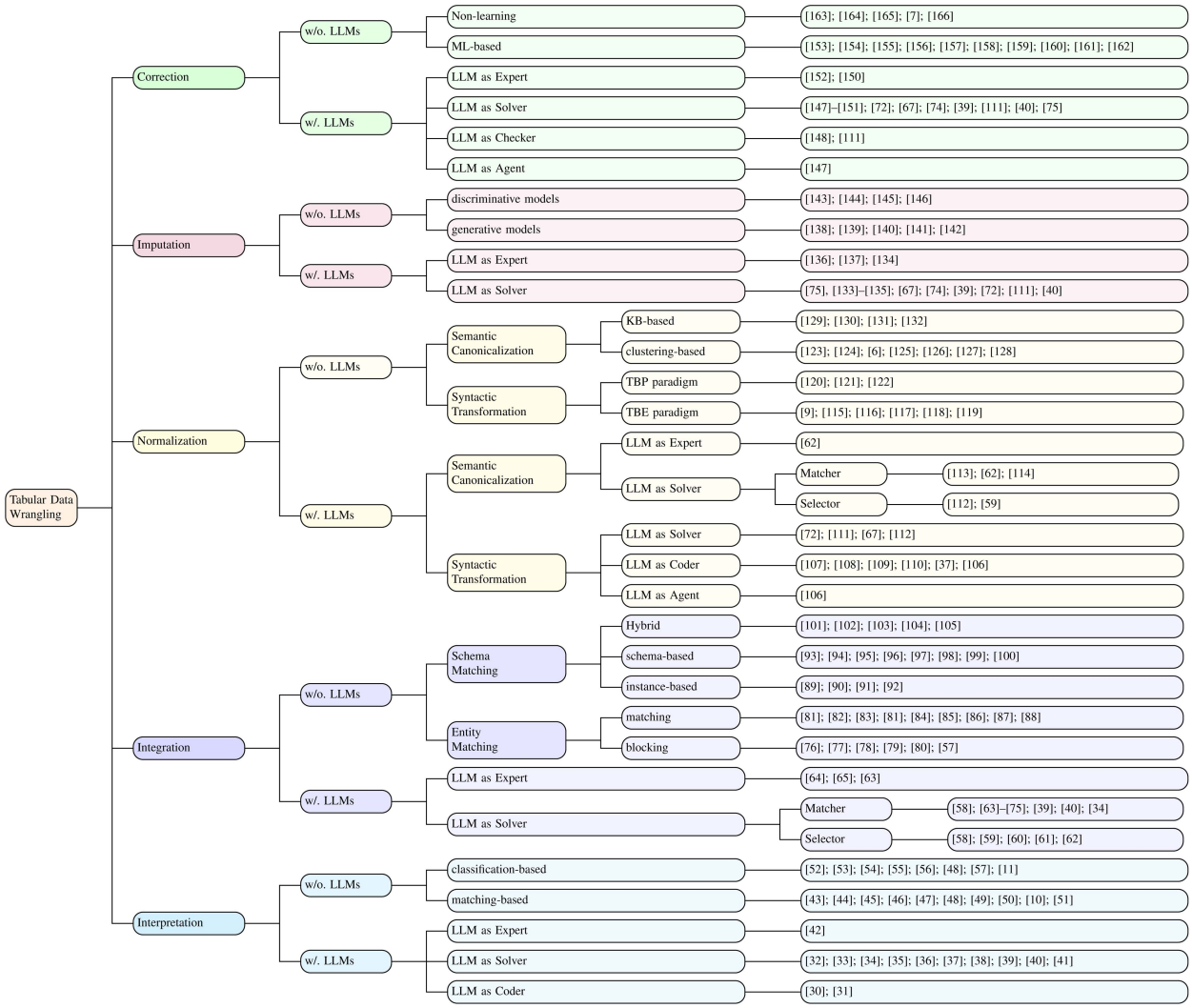


Fig. 2. A taxonomy for Tabular data Wrangling techniques in the Era of Large Language Models (since 2010).

Definition 4. (Record): A record r_j corresponds to the j-th row in table T, consisting of a set of values: $\{v_{ji}\}_{1\leq i\leq n}$, where n is the number of attributes (columns) in T.

Definition 5. (Schema): The schema S of table T defines its structural metadata, including column names, data types, and constraints. In most cases, we consider $S = A$.

2) Tabular Data Wrangling Tasks: Data wrangling, also known as *data munging*, is the process of cleaning, structuring, and transforming raw data into a usable, concise, and intelligible format. In the context of tables, data wrangling process encompasses five key tasks: correction (Fig. 1(a)), imputation (Fig. 1(b)), normalization (Fig. 1(c)), integration (Fig. 1(d)) and interpretation (Fig. 1(e)). The first three tasks are closely related to data quality, which have been activate research areas for decades. The latter two tasks play a crucial role in downstream applications such as data analysis and decision-making. *(a) Table Correction* (TC) focuses on identifying and repairing incorrect values within a table. It typically involves two steps: (1) error detection, and (2) data repairing. Typical errors include factual errors (e.g., "December 32nd, 2019"), typographical mistakes (e.g., "cicago"), inconsistencies (e.g., a product priced at 40 in Table A but 50 in Table B), and logical constraint violations (e.g., a staff member's departure date preceding their entry date).

Definition 6. (Table Correction): Given a table $T = (A, V)$, the goal of table correction is to identify all incorrect data cells $X' \subset V$ and predict correct value x_i^* for each $x_i \in X'$.

(b) Table Imputation (TIp) aims to fill missing values in a table. Such values are common in real-world datasets and may introduce bias or reduce reliability of downstream analyses.

Definition 7. (Table Imputation): Given an incomplete table $T = (A, V)$ and a set of data cells with missing values $\tilde{X} \subset V$, the goal of table imputation is to predict the most appropriate value x_i^* for each $x_i \in \tilde{X}$.

(c) Table Normalization (TN) aims to reduce redundancy and ambiguity of table values. It includes two sub-tasks: *Semantic Canonicalization* (SC) and *Syntactic Transformation* (ST). Semantic canonicalization groups different expressions of the same entity or conveying similar meanings (e.g., "Trump" and "Donald Trump"; "doctorate" and "Ph.D."), while Syntactic transformation unifies value formats within a column (e.g., formatting dates as "YYYY-MM-DD").

Definition 8. (Semantic Canonicalization): Given a column $C_i = \{v_{ji}\}_{1\leq j\leq m}$ in table T and its corresponding attribute a_i,

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

the goal of semantic canonicalization is to group values referring to the same real-world entity (if v_{ji} is entity-based) or cluster values with similar meaning (if v_{ji} is literal-based).

Definition 9. (Syntactic Transformation): Given a column $C_i = \{v_{ji}\}_{1 \le j \le m}$ in table T with its corresponding attribute a_i, syntactic transformation aims to standardize value formats or unify measurement units within C_i.

(d) Table Integration (TInt) aims to merge information from one or more sources into a unified representation. It requires establishing correspondences between data elements, including equivalence ($=$), containment ($\in$ or $\subset$), disjointness ($\perp$), and overlap ($\cap$). Among them, equivalence is the most widely studied and typically referred to as table matching.

Definition 10. (Table Matching): Given two sets of data elements $E = \{e_1, \ldots, e_m\}$ and $E' = \{e'_1, \ldots, e'_m\}$ from one or more tables, the goal of table matching is to identify all matched pairs $(e_j, e'_j) \in E \times E'$ that are considered "same" in the real world.

The data elements involved in table matching includes rows (records) and columns. Accordingly, table matching includes two sub-tasks: (1) *Entity Matching* (EM): identifying equivalence between rows or records, as shown at the top of Fig. 1(d); (2) *Schema Matching* (SM): identifying equivalence between columns or attributes, as shown at the bottom of Fig. 1(d).

(e) Table Interpretation (TItp), also known as *Semantic Table Annotation*, focuses on uncovering the semantics of data elements within a table and converting them into machine-readable knowledge [167]. This process encompasses multiple levels of annotation: 1) *Cell Entity Annotation* (CEA), identifying mentions within individual data cells and linking them to corresponding entities in a referenced Knowledge base (KB). 2) *Record Entity Linking* (REL), linking entire records (i.e., rows) to real-world entities in a referenced KB. 3) *Column Type Annotation* (CTA), associating a column with a semantic type, such as concepts (e.g., "country") or literal value types (e.g., "date"). 4) *Column Property Annotation* (CPA), identifying the semantic relationships between two columns in a table, such as "datePublished" or "birthday".

Definition 11. (Cell Entity Annotation): Given a table T with a set of data cells V, and a KB containing a set of entities E, the goal of cell entity annotation is to learn a mapping function $f : V \to E$ that assigns each cell $x_i \in V$ to its most appropriate entity $e_j \in E$, if such an entity exists.

Definition 12. (Record Entity Linking): Given a table T with a set of records R, and a KB containing a set of entities E, the goal of record entity linking is to find a mapping function $f : R \to E$ that links each record $r_i \in R$ to the entity $e_i \in E$ it represents.

Definition 13. (Column Type Annotation): Given a table T with a set of columns $\{C_i\}_{1 \le i \le n}$, and a set of types L defined in an ontology or a KB, the goal of column type annotation is to assign a label $l_i \in L$ that best describes the semantic type of each column C_i.

Definition 14. (Column Property Annotation): Given a table T with a set of columns $\{C_i\}_{1 \le i \le n}$, and a set of properties P defined in a KB, the goal of column property annotation is to identify a property $p_i \in P$ that best describes the relationship between a pair of columns $(C_i, C_j)_{i \ne j}$, where C_i is the subject column and C_j is the object column.

It is worth noting that the existing literature does not consistently draw a clear distinction between *Cell Entity Annotation* and *Record Entity Linking*. Accordingly, we adopt the same approach in this survey and treat them interchangeably.

B. LLM-Free Tabular Data Wrangling Methods

In this section, we provide a general overview of approaches that do not rely on LLMs for tabular data wrangling.

a) Table Correction: Errors and inconsistencies in tables can lead to misleading analyses and erroneous decisions. Based on their reliance on training data, LLM-free correction methods can be broadly divided into two groups: 1) Non-learning methods [7], [163], [164], [165], [166], which rely on manually defined rules (e.g., denial constraints), crowd-sourcing techniques (e.g., providing user-assisted visualization tools for error identification), or external data sources (e.g., validated address listings); 2) Machine Learning (ML)-based methods [153], [156], [157], which often depend on feature engineering. To reduce labeling cost, some methods [154], [155], [158] use active learning, while others adopt semi-supervised learning [159], [162], self-supervised or Bayesian inference-based strategies [160], [161] to improve generalization under limited or no manual labels.

b) Table Imputation: Data may be missing for a variety of reasons, such as accidental omission, acquisition cost, or safety concerns during data collection. State-of-the-art imputation methods are generally divided into discriminative and generative models. Discriminative models predict missing values directly from observed data. For example, some studies [143], [144] train predictive models on complete records, while others [145], [146] apply matrix decomposition techniques to approximate missing entries by leveraging the low-rank structure of the data matrix. In contrast, generative models reconstruct missing entries by modeling the joint distribution of observed and missing values. Representative examples include DAEs (Denoising Autoencoders for short)-based methods [138], [141], which treat missing values as noise, GAN (Generative Adversarial Networks for short)-based methods [139], [140], which use a generator and discriminator to impute and distinguish missing values, and other methods based on Variational Autoencoders (VAEs) [141] or Diffusion Models [142].

c) Table Normalization: Tables often contains redundant entries and non-standard expressions, which hinder effective analysis. Table normalization mainly involves two sub-tasks: semantic canonicalization and syntactic transformation. *Semantic Canonicalization (SC)* groups semantically equivalent mentions into a unified form. Existing methods either rely on additional KBs to link mentions to curated KB entries and then group those mapped to the same entity or relation [129], [130], [131], [132], or formulate the task as clustering over manually designed features or learned embeddings [6], [123], [124]. *Syntactic Transformation (ST)* aims to convert values into a consistent format. Existing methods include Transform-by-Example (TBE), which synthesizes transformation programs from user-provided input-output examples [9], [115], [116], [117], [118], [119], and Transform-by-Pattern (TBP), which derives transformation programs from input-output patterns without explicit examples [120], [122]. Since many real-world tables contain hetero-

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

geneous value formats within a column, USTL [121] further proposes an unsupervised strategy that clusters value pairs with similar transformation logic for more flexible normalization.

d) Table Integration: Table integration mainly involves two sub-tasks: Entity Matching and Schema Matching. *Entity Matching (EM)*, also known as *Entity Resolution* or *Record Linkage*, aims to identify records referring to the same real-world entity. Due to the potentially large number of records, EM is typically conducted in two stages: blocking and matching. Blocking reduces the cost of pairwise comparisons by filtering unlikely pairs, and commonly includes sorting-based approaches that group records by shared keys or hash value [76], [77], and ranking-based methods that retrieve candidate pairs based on similarity scores or top-k neighbors [57], [78], [79], [80]. Matching then verifies whether each candidate pair refers to the same entity. Existing methods are mainly learning-based, including supervised [82], [86], [87], semi-supervised [81], [83], [88], and unsupervised [84], [85].

Schema Matching identifies semantically equivalent attributes across tables. Compared with EM, it usually involves far fewer candidates and thus does not require blocking. Existing LLM-free methods can be broadly divided into instance-based, schema-based, and hybrid methods. Instance-based methods exploit attribute values and metadata such as value distributions or overlaps [89], [90], [91], [92]. Schema-based methods rely on schema-level features such as attribute names, data types, and headers [93], [94], [95], [96], [97], [98], [99], [100]. Early studies mainly used handcrafted similarity metrics [100], while recent methods adopt deep learning models for richer semantic representations [93], [94]. Hybrid methods combine instance- and schema-level signals to overcome the limitations of using either alone [101], [102], [103], [104], [105]. Some recent studies further formulate schema matching as a similarity matrix refinement problem using neural models or human feedback [98], [99].

e) Table Interpretation: Based on how the interpretation task is formulated, LLM-free methods can be broadly categorized into two groups. Classification-based methods [11], [48], [52], [53], [54], [55], [56], [57] treat interpretation as a classification problem, where each table element, such as a cell, column, or row, is assigned a label from a predefined set (e.g., semantic types or column roles). In contrast, matching-based methods [10], [43], [44], [45], [46], [47], [48], [49], [50], [51] formulate interpretation as a table-to-knowledge-base (KB) alignment problem, aiming to match table elements (e.g., columns, cells, or column pairs) with corresponding entries in a curated KB. These two paradigms reflect differing assumptions and objectives: classification-based methods emphasize label prediction accuracy within a closed set, while matching-based approaches focus on linking tabular content to external structured knowledge.

Discussion: LLM-free methods for tabular data wrangling, including deep learning- and rule-based approaches, have shown strong performance across tasks. However, they also exhibit notable limitations in complex real-world settings. First, constrained by model capacity and task-specific training data, they often struggle to capture intricate patterns and generalize to unseen datasets or new tasks. Second, they are particularly vulnerable to dirty data, such as mislabels, typos, and missing values, which are common in real-world tables. Finally, they often require substantial manual effort to design task-specific features, rules, or models, leading to high time and expertise costs.

C. Table Understanding and Reasoning

Although this survey focuses on tabular data wrangling, table understanding and reasoning are closely related research problems. Unlike wrangling, which aims to improve data quality and usability, these tasks focus on enabling models to understand table structures and perform logical, numerical, and multi-hop reasoning over tables, supporting downstream applications such as Table QA, fact verification, and spreadsheet manipulation. These abilities are also useful for wrangling, e.g., cross-table inconsistencies detection, conflict resolution during integration, and reasoning over integrated tables.

Existing methods can be broadly grouped into two categories. The first is pre-trained table-specific methods, which design dedicated pre-training or fine-tuning framework for tables. Representative works include TAPAS [12] and TAPEX [168], which learn table representations from large-scale table-text corpora. While effective at capturing table structure and semantics, these methods often general poorly to unseen domains, new schemas, and complex reasoning scenarios. The second is LLM-based methods, which leverage the in-context learning and reasoning abilities of LLMs for more flexible table processing. For example, Dater [169] treats LLMs as decomposers that break down tables and questions into sub-evidence and sub-queries; Chain-of-Table (CoT) [170] performs reasoning by iteratively generating operations to update tables; and Tree-of-Table (ToT) [171] further improves reasoning over large tables via hierarchical organization. More recent studies enhance LLMs with additional mechanisms to improve scalability and reliability, such as hypergraph neural networks in HyperG [172], external table retrieval in OpenTab [173], and plan-then-execute reasoning in PoTable [174].

III. Datasets and Benchmarks

Traditional datasets, such as Hospital [7], Beers,[1] Movies [216], and ODDS [217], have been widely used for evaluating tabular data wrangling techniques. While these benchmarks have played an important role in advancing single-table cleaning methods [7], [100], they exhibit several fundamental limitations. First, they primarily assume that data quality issues occur within isolated single tables, overlooking cross-table dependencies and error propagation that arise in multi-table settings. In practice, correcting one issue in a table may inadvertently introduce new inconsistencies either within the same table or across related tables. Second, the evaluation protocols based on these datasets are often limited to static, single-step cleaning performance (e.g., cell-level accuracy or F1), without considering end-to-end workflow effectiveness or downstream task impact.

[1] https://www.kaggle.com/datasets/nickhould/craft-cans

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

TABLE II
OVERVIEW OF EXISTING DATASETS AND BENCHMARKS FOR TABULAR DATA WRANGLING. WE INCLUDE ONLY DATASETS WITH MORE THAN FIVE TABLES THAT HAVE BEEN USED BY MORE THAN ONE RELEVANT METHOD. ACC STANDS FOR ACCURACY. F1 IS CALCULATED FROM THE PRECISION AND RECALL, AND SIMILARITY REFERS TO THE SEMANTIC DISTANCE BETWEEN TOKEN EMBEDDINGS.

Dataset	Domain	#Tables	Tasks	Evaluation Metric	Papers that used this data
T2Dv2 [175]	General	779	TItp (CPA, CEA)	F1	[33], [45], [176]
GitTables [177]	General	1,101	TItp (CTA, CPA, CEA)	F1	[36], [178], [179]
2T [180]	General	180	TItp (CTA, CEA)	F1	[181]–[183]
BioDivTab [184]	Biodiversity	50	TItp (CTA, CEA)	F1	[185]–[187]
SOTAB [188]	General	107,927	TItp (CTA, CPA)	F1	[32], [33], [36]
Hard Tables [189]	General	8,957	TItp (CTA, CPA)	F1	[181], [190]
WikiTable [191]	General	580,171	TItp (CTA, CPA)	F1	[48], [192], [193]
REDTab [194]	General	9,149	TItp (CPA)	F1	[195]
REIN [157]	Multi-domain	14	TC, TIp	F1	[148], [152], [196]
AutoDCWorkflow [147]	Multi-domain	96	TC	F1, similarity	[197], [198]
MammoTab [199]	General	838,930	TItp (CTA, CEA)	-	[200]–[202]
WDC Products [203]	E-commerce	-	TInt (EM)	F1	[58], [63], [204]
FEVEROUS [205]	General	28,800	TC	Acc	[206]–[208]
S2abEL [209]	Scientific	732	TItp (CEA)	Acc	[42], [210], [211]
SSB [212]	Business	5	TC	F1	[213]–[215]

To address these limitations, recent benchmarks (as summarized in Table II) have been proposed to better reflect real-world tabular data wrangling scenarios. For example, GitTables [177] collects millions of CSV tables from GitHub repositories and aligns them with external knowledge bases. These tables originate from real software projects, providing broader topical coverage and more realistic data distributions. Similarly, Hard Tables [189] focuses on challenging real-world cases, containing a large number of ambiguous and noisy cells that are difficult to resolve unambiguously. As a result, traditional table wrangling methods experience significant performance degradation on this benchmark, highlighting the gap between controlled experimental settings and real-world complexity. Despite these advances, current benchmarks still exhibit notable limitations. Many datasets are derived from a relatively narrow set of sources, such as Wikipedia and web tables, and therefore lack sufficient diversity in domain-specific settings, including biomedical, financial, and industrial data. In addition, some recent datasets are partially synthesized or augmented using LLMs, which may introduce distributional difference and fail to capture the long-tail noise patterns observed in real-world data. Furthermore, existing evaluation protocols remain largely task-specific and do not adequately assess the effectiveness of multi-step, cross-table wrangling workflows.

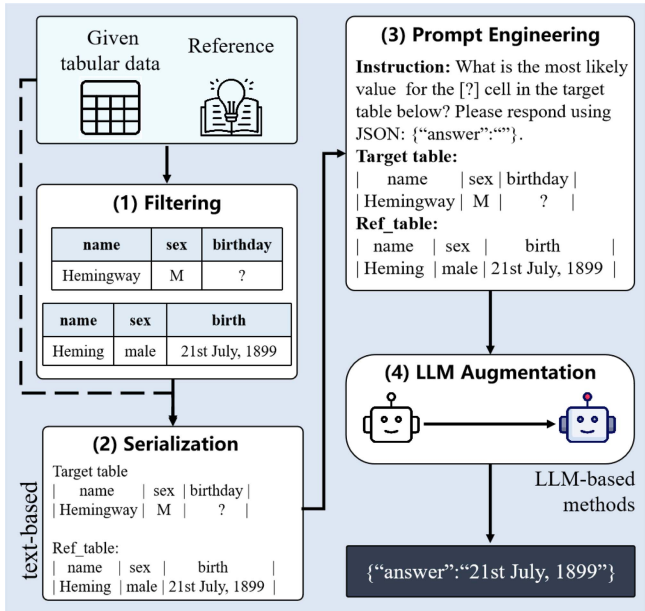


Fig. 3. Key techniques for LLM-based tabular data wrangling. (1) **Filtering** reduces the input length while preserving core information; (2) **Serialization** converts tables into text acceptable by LLMs; (3) **Prompt Engineering** guides LLMs to perform tabular tasks and generate reliable outputs without updating parameters; (4) **LLM Augmentation** integrates external tools or training strategies to enhance LLM performance.

IV. KEY TECHNIQUES IN LLM-BASED TABULAR DATA WRANGLING WORKFLOWS

In this section, we identify four key modules for applying LLMs to tabular data wrangling, as illustrated in Fig. 3.

A. Filtering

Real-world tables often consist of tens of thousands of rows and hundreds of columns. When linearized into text for LLM processing, their length can vastly exceed the input limits of most models, such as 16 K tokens for ChatGPT [16] and 128 K tokens for LLaMA3.1 [17]. Excessive input length not only increases computational cost but also significantly degrades model performance [218], largely due to the inclusion of redundant or irrelevant information. Consequently, table segmentation and filtering become critical preprocessing steps when leveraging LLMs for tabular tasks. In particular, effective filtering, removing noise while retaining essential content, is crucial for enhancing both efficiency and accuracy. To mitigate these challenges, GPT4Table [219] imposes predefined constraints to keep inputs within acceptable limits, while other approaches [133], [149] adopt selective filtering strategies that retain only the most relevant table elements (e.g., rows, columns, or cells), as illustrated in Fig. 3(1).

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

TABLE III
TEXT-BASED SERIALIZATION METHODS: AN EXAMPLE OF 2×3 TABLE, WHERE a_i DENOTES THE HEADER OF i-TH COLUMN, AND v_{ji} DENOTES THE CELL VALUE AT j-TH ROW AND i-TH COLUMN

Method	Description	Examples	Papers
Markdown	line-separates rows and separates columns by "|"	| a_1 | a_2 | a_3 | |------|------|------| | v_{11} | v_{12} | v_{13} | | v_{21} | v_{22} | v_{23} |	[220] [39] [221]
Vanilla-T	lists headers followed by cells in each row sequentially	$a_1, a_2, a_3, v_{11}, v_{12}, v_{13}, v_{21}, v_{22}, v_{23}$	[222]
Data Matrix	transforms the table as a list of lists	[[a_1, a_2, a_3], [v_{11}, v_{12}, v_{13}], [v_{21}, v_{22}, v_{23}]]	[220]
Row-Identifier	adds an identifier as the prefix for each row to distinguish different rows in the linearized table sequence	[HEAD] a_1 | a_2 | a_3, [ROW1] v_{11} | v_{12} | v_{13}, [ROW2] v_{21} | v_{22} | v_{23}	[41], [168]
Column-Identifier	adds an identifier as the prefix for each column to distinguish different columns in the linearized table sequence	[COL1] a_1 | v_{11} | v_{21}, [COL2] a_2 | v_{21} | v_{22}, [COL3] a_3 | v_{31} | v_{32}	[14]
LaTeX	separates rows by "\\$hline$" and separates columns by "&"	\\hline v_{11} & v_{12} & v_{13} \\\\ \\hline v_{21} & v_{22} & v_{23} \\\\	[223] [224]
X-Separated	line-separates rows and separates columns by ",", "\t", ";", etc	a_1, a_2, a_3 v_{11}, v_{12}, v_{13} v_{21}, v_{22}, v_{23}	[220]
Attr-Value Pairs	concatenate paired columns and cells in the format "a:v"	$a_1 : v_{11}$; $a_2 : v_{12}$; $a_3 : v_{13}$	[72], [225]
HTML	adds HTML element for tabular data	<table><thead><tr><th> a_1</th> <th> a_2</th><th> a_3</th></tr></thead> <tbody><tr><td> v_{11}</td><td> v_{12}</td> <td> v_{13}</td> ... </tbody></table>	[220]
DFLoader	loads the table as a Pandas dataframe	pd.DataFrame({ a_1 : [v_{11} , v_{21}], a_2 : [v_{12} , v_{22}], a_3 : [v_{13} , v_{23}]})	[220]
Values-only	list cell values in one row/column sequentially	v_{11}, v_{12}, v_{13}	[63]
Row_sentence	converts the row into a sentence with different templates	a_1 is v_{11} , a_2 is v_{12} , a_3 is v_{13}	[224], [226] [223]
Colume_sentence	converts the column into multiple sentences with the same template	a_1 is v_{11} , a_1 is v_{21} .	[151]
Row-JSON	represents each row as a JSON, where the column names and their corresponding cell values are represented as key-value pairs	[{0: { a_1 : v_{11} , a_2 : v_{12} , a_3 : v_{13} }}, {1: { a_1 : v_{21} , a_2 : v_{22} , a_3 : v_{23} }}]	[220]

Tree-JSON	converts the table into a tree, where each cell's hierarchical structure, position information, and content is represented as a tuple	{"top_root": {
		"value": "<TOP>",
		"children_dict": [{
		"value": a_1,
		"children_dict": []
		},
		...]},
		"data": [[
		{"value": v_{11}},
		...], [
		{"value": v_{21}},
		...]]
		}
			[227]

B. Serialization

Since LLMs are inherently designed to process linear text, a fundamental challenge in applying them to tabular tasks is the conversion of inherently structured, two-dimensional tables into flat textual representations. Effective serialization must preserve the semantic and structural integrity of the table, capturing column-wise relationships, row-wise context, and inter-cell dependencies, since this information is crucial for downstream tasks such as cell classification, entity linking, and row/column ranking. A range of encoding strategies has been proposed for integrating tabular data into neural models, including text-based, graph-based, and image-based approaches. In this survey, we focus specifically on *text-based serialization methods*, as our primary interest is in leveraging LLMs for tabular tasks. Table III provides a summary of common text-based serialization strategies used in recent studies.

Comparisons: Prior studies have investigated the impact of different serialization strategies on the performance of LLMs across a variety of tabular tasks. Overall, structured formats (e.g. Markdown, JSON, and HTML) lead to promising improvements in LLM performance on various tabular tasks, compared to natural language serializations (e.g., sentence, X-Separated) [179]. Among them, structured formats like DFLoader and JSON have demonstrated superior effectiveness in tasks involving Fact-Finding and Data Transformation [220]. In contrast, formats like HTML or XML have proven more effective for tasks, like Table Question Answering (Table-QA) and Table Fact Verification (Table-FV), than JSON or Markdown [179].

C. Prompt Engineering

LLMs typically operate through a prompt-response interface, where an *instruction* guides the model to produce a *completion*, as illustrated in Fig. 4. At its core, an instruction comprises a task description and a task input. However, conventional prompts, often focus on a single row or column, are insufficient for complex tabular tasks that require access to broader table context. To address this limitation, Table-GPT [39] introduces a triplet prompt structure $(Instruction, Table, Completion)$, as shown in Fig. 5.

Beyond prompt format, several training-free strategies have been proposed to improve LLM performance [228]. Few-shot learning provides task demonstrations to help the model infer the

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

Instruction	Role (optional)	You are a seasoned expert in the field of academic classification and identification.
	Task Description	**This is a deduplication task.**
	Result Format (optional)	Provide your ultimate answer as 'Yes' or 'No'.
	Knowledge (optional)	Note that mising values (N/A or "nan") should not be used as a basis for your dicision. To determine if two values are identical, you need to examine both their full names and corresponding acronyms.
	Task Demonstration (optional)	Record 1: [Publisher: "Little, Brown Books for Young Readers", Author: "Darren Shan", Title:"Trials of Death"] ; Record 2: [Author: "Darren Shan", Title:"Birth of a Killer",Publisher: "Little, Brown Books for Young Readers",] Answer: No.
	Task Input	**Record 1: [Publisher: "Disney-Hyperion", Author: "Gordon Korman", Title:"The Juvie Three"] ; Record 2: [Publisher: "Hyperion Books", Author: "Gordon Korman", Title:"The Juvie Three"]**
Completion	**Task Output**	**Answer: __**

Fig. 4. A common prompt format for leveraging LLMs: $(Instruction, Completion)$ paradigm.

Instruction	**Task Description**	**The cell marked as [TARGET] in the table below is missing, what value should be there?**
	Result Format (optional)	Use JSON {"answer": ""}
Table		**| row-id | Country | Continent | GPD | ... | row-1 | USA | Americas | 26,854,599 | ... | row-2 | China | [TARGET] | 19,373,586 | ...**
Completion	**Task Output**	**China is in Asia, so it is{"answer": "Asia"}**

Fig. 5. A special prompt format for leveraging LLMs in tabular tasks: $(Instruction, Table, Completion)$ paradigm.

task pattern, though its effectiveness depends heavily on example similarity, order, and diversity [63], [66], [75], [133], [229], [230]. Retrieval-augmented generation (RAG) incorporates relevant external knowledge to overcome knowledge limitations and mitigate hallucination issues [69]. Reasoning-augmented prompts, such as CoT and ToT, further improve performance by encouraging explicit multi-step reasoning, and have shown effectiveness in tabular tasks [32], [35].

D. LLM Augmentation Techniques

While vanilla LLMs can be applied to tabular tasks via prompt engineering, they suffer from key limitations, including hallucination, and static internal knowledge. Moreover, the unique structure of tables poses additional challenges. Unlike plain text, tables (1) are inherently two-dimensional; (2) require both row-wise and column-wise interpretation; and (3) are often semantically invariant to row or column reordering. These properties conflict with the sequential nature of LLMs.

Beyond structural complexity, tables introduces other challenges. First, *heterogeneity*: tables often contain diverse data types (e.g., categorical, numerical, binary, and text), making unified modeling difficult. Second, *sparsity*: real-world tables frequently contain missing values and long-tailed distributions, hindering reliable pattern learning. Third, *contextual knowledge*: tables encode implicit relationships and domain-specific constraints (e.g., age requirements), which are difficult for general-purpose LLMs to capture.

To address these limitations, recent research has explored a variety of augmentation techniques (Fig. 6), including role-based frameworks where multiple LLMs collaborate to improve robustness and accuracy. In addition, two training-based strategies are commonly adopted. *Continual pre-training (CPT)* enhances structural understanding by training on table-centric corpora. For example, GTL [231] converts tables into instruction-style data for pre-training, while TableGPT2 [232] improves stability and efficiency through mixed-precision training and length-aware batching. *Supervised fine-tuning (SFT)* further improves task performance by adapting LLMs to specific tabular tasks, either using labeled data or distilling reasoning from larger models [40], [64], [65], [178]. Advanced frameworks, such as Low-Rank Adaptation (LoRA) [233] used in Recommender [234], LongLoRA in TableLLaMA [41], T-Few [235] in TabLLM [224], and auxiliary-task-enhanced training in CODDLLM [236], further strengthen LLM capabilities on tables.

V. LLM-Based Methods for Tabular Data Wrangling

In this section, we categorize existing LLM-based methods for tabular data wrangling based on both the specific tasks they address and the roles that LLMs assume within these methods. As illustrated in Fig. 2, LLMs typically play the following five roles: (1) **Solver**, directly generating the final output for a task; (2) **Expert**, providing relevant background knowledge or domain-specific insights; (3) **Coder**, producing executable code (e.g., SQL, Python) to solve a given problem; (4) **Checker**, validating the correctness or consistency of outputs; and (5) **Agent**, coordinating multiple tools or modules to obtain the final result. In addition to role-based categorization, we analyze differences across several other dimensions, including application domains, table serialization strategies, and the integration of external knowledge sources, as summarized in Table IV.

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

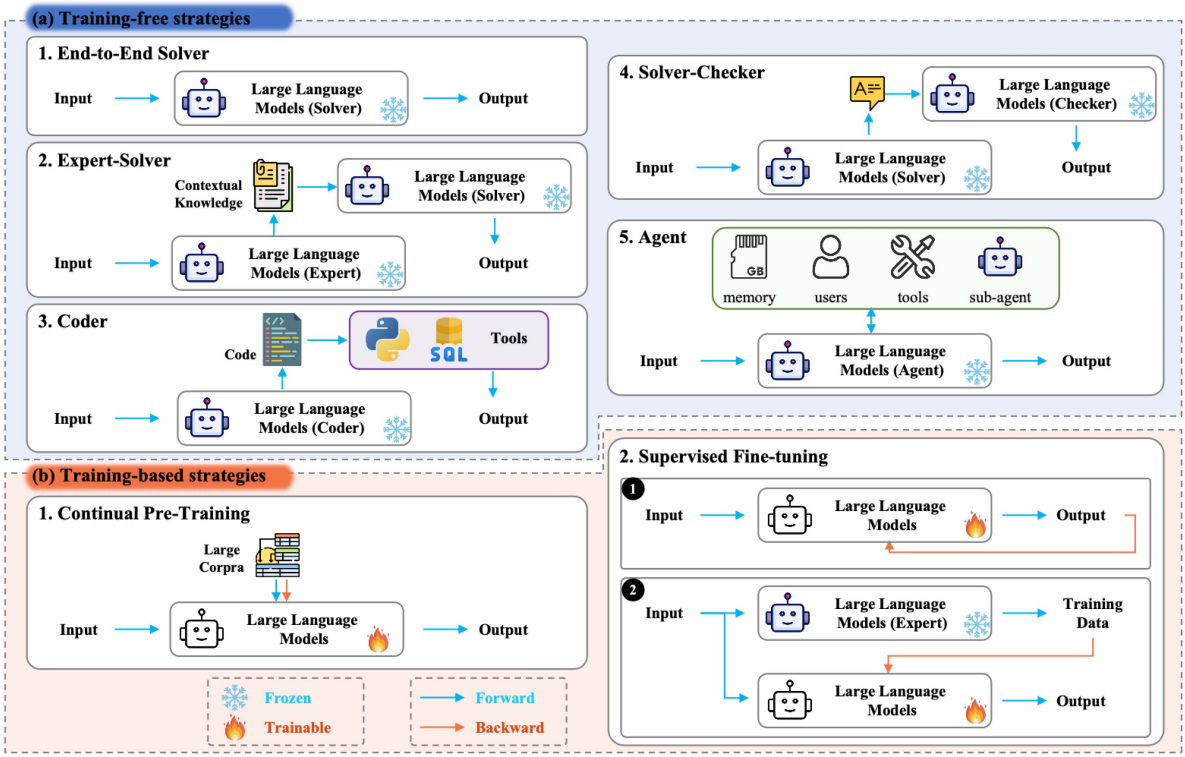


Fig. 6. Two strategies are used to enhance LLMs for tabular data wrangling: (a) **Training-free**, which leverage vanilla LLMs without additional training; (b) **Training-based**, which improve LLMs through continual pre-training or supervised fine-tuning on tabular data. Training-free methods can be further grouped into five paradigms: (1) End-to-end solver, where the LLM directly produces the final result; (2) Expert-solver, where LLMs provide contextual information that guides downstream generation; (3) Coder, where LLMs generate executable codes to obtain the final result; (4) Solver-checker, where an LLM verifies the correctness of the result generated by LLMs; and (5) Agent, where LLMs orchestrate multiple specialized modules to collaboratively solve complex tasks.

A. *Table Correction*

Recent research on table correction can be broadly categorized based on the role played by the LLM: LLM as Expert, LLM as Solver, and LLM as Checker.

1) LLM as Expert: Traditional correction methods often rely on static signals, such as integrity constraints, user-defined rules, or external knowledge bases. While these approaches can be effective in controlled environments, they typically require extensive manual effort to capture implicit relationships. Moreover, such systems often lack scalability and may be brittle when rules conflict or fail to generalize. To address these limitations, recent studies leverage LLMs as experts to extract functional dependencies (FDs), particularly Ontological Functional Dependencies (OFDs), to guide the correction process [150], [152]. For example, LLMClean [152] uses LLMs to derive context models from raw data without requiring additional metadata, and introduces novel dependency types, such as Sensor Capability Dependencies and Device-Link Dependencies, to enhance error detection. Similarly, Cocoon [150] simulates human-in-the-loop cleaning processes by using LLMs to verify whether statistically strong FDs also align with semantic expectations.

2) LLM as Solver: LLMs can also function as solvers, directly identifying and correcting erroneous values using the knowledge encoded in their parameters. In this role, LLMs are prompted to detect anomalies and propose corrections. For instance, foundation models like GPT-3 have been queried with prompts such as "Is there an error in Country? Country: England, City: Kyoto?" to detect inconsistencies [72]. Besides, few-shot prompting has been shown to outperform zero-shot approaches and traditional baselines in such tasks. Cocoon [150] also employs LLMs to map typographical errors to corrected values and applies CASE-WHEN clauses for automated repairs. IterClean [148] further divides the solver role into two sub-roles: a detector that identifies errors with accompanying explanations, and a repairer that suggests context-aware corrections. Recognizing the limitations of LLMs, particularly with outdated or incomplete knowledge, some studies introduce retrieval-augmented correction frameworks. For example, RetClean [149] retrieves the top-k relevant tuples from a user-defined data lake using ElasticSearch and Faiss, while UNIDM [67] uses an LLM to locate external reference data and another LLM to perform correction based on the retrieved context. Additional work [40], [151] try to improve LLM correction performance on domain-specific datasets by fine-tuning.

3) LLM as Checker: Conventional detect-then-repair pipelines often fail to resolve all errors and may inadvertently introduce new ones [196], [237]. To mitigate this, recent studies adopt iterative frameworks that use LLMs as checkers to verify and refine corrections over multiple rounds. For example, IterClean [148] inserts a verifier LLM between the detector and repairer. This verifier validates error claims by cross-referencing

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
LLM-BASED METHODS FOR VARIOUS TABULAR DATA WRANGLING TASKS

Reference	Tasks	Domain	vanilla LLMs	Role of LLMs	Serialization	Extra Knowledge
ArcheType [36]	CTA	general	LLaMA-7B, Flan-T5/UL2, GPT-3.5/4	Solver	only values	✗
evaluation [35]	CTA	general	LLaMA3-70B, GPT-3.5	Solver	only values	✗
RACOON [38]	CTA	general	GPT-3.5	Solver	only values	✓
2-step pipeline [32]	CTA	general	GPT-3.5	Solver	markdown	✗
CHORUS [34]	CTA, SM	general	ChatGPT	Solver	markdown	✓
FST [37]	CTA, ST	general	GPT-4	Solver	only values	✗
DAGOBAH SL [30]	CTA,CEA,CPA	general	Codex	Coder	markdown	✗
FT+ICL [33]	CTA,CPA	general	GPT-3.5/4, SOLAR-70B[2]	Solver	markdown	✗
synthetic [42]	CEA	scientific	GPT-3.5/4, TULU2-70B	Expert	only values	✓
pool-search [133]	CEA,TIp,TC	general	FLAN-T5-3B, Vicuna-7B	Solver	attr-value pairs	✗
COMEM [58]	EM	general	GPT-3.5/4o-mini, LLaMA3.1-8B, Mistral-7B/8*7B, Flan-T5/UL2, Solar-10.7B, Command-R-35B	Matcher, Selector	attr-value pairs	✗
Rules+ICL [63]	EM	general	GPT-4/4o(-mini), LLaMA2-70B, LLaMA3.1-70B, Mixtral-8*7B	Expert, Matcher	attr-value pairs	✗
BATCHER [66]	EM	general	GPT-3.5/4, LLaMA2-70B	Matcher	attr-value pairs	✗
MQsSP [68]	EM	general	GPT-4	Matcher	only values	✗
GRAM [61]	SM	general	GPT-3, T0++, Flan-UL2	Selector	only attributes	✗
KCMF [70]	EM,SM	general	GLM-4, LLaMA3-8B, Mistral-7B, GPT-3.5/4o	Matcher	sentence	✗
EDC [62]	EM,SM,SC	general	GPT-3.5/4, Mixtral-7B	Expert, Matcher	-	✗
FM-based [72]	EM,TIp,TC,ST	general	GPT-3-175B	Solver	attr-value pairs	✗
data preprocessor [74]	EM,TIp,SM,TC	general	GPT-3.5/4/4o	Solver	attr-value pairs	✗
UniDM [67]	EM,TIp,TC,ST	general	GPT-3-175B, GPT-J-6B, LLaMA2-7B	Solver	attr-value pairs	✓
prompt [73]	EM	e-commerce	GPT-3.5	Matcher	attr-value pairs	✗
auto-prompt [75]	EM,TIp,TC	general	Mixtral8*7B, Flan-t5-xxl	Solver	-	✗
RAG-based [69]	SM	general	Jellyfish-8B, GPT-4o-mini	Matcher	-	✓
Autoalign [59]	SM	general	Claude	Selector	-	✗
Harmonia [60]	SM	general	-	Selector	-	✗
few-shot clustering [113]	SC	general	GPT-3.5	Matcher	-	✗
k-Means-based [114]	SC	scientific	GPT-3	Matcher	-	✗
WDC-PAVE [112]	SC,ST	general	GPT-3.5/4	Solver	-	✗
Normtab [107]	ST	general	GPT-3.5	Coder	-	✗
SQLMorpher [108]	ST	general	GPT-3.5/4	Coder	only values	✗
CleanAgent [106]	ST	general	-	Coder, Agent	-	✗
DATAMORPHER [110]	ST	general	-	Coder	data matrix	✗
IDSM+TRGM [109]	ST	IoT	GPT-4	Coder	-	✗
MLR+LLM [136]	TIp	general	ChatGPT	Expert	-	✗
CLAIM [137]	TIp	general	LLaMA2-7B	Expert	-	✗
prompt [134]	TIp	scientific	GPT-4/4o-mini, LLaMA3-8B	Expert, Solver	-	✗
LLM-Forest [135]	TIp	general	GPT-4o, Claude-3.5, Mixtral-8×22B	Solver	attr-value pairs	✗
LLMClean [152]	TC	IoT	LLaMA2-7B/13B/70B, GPT3.5/4	Expert	only attributes	✗
Cocoon [150]	TC	general	Claude 3.5	Expert, Solver	only values	✗
IterClean [148]	TC	general	GPT-3.5	Solver, Checker	-	✗
AutoDCWorkflow [147]	TC	general	LLaMA3.1-7B, Mistral-8B, Gemma2-27B	Solver	markdown	✗
RetClean [149]	TC	general	ChatGPT	Solver	attr-value pairs	✓
batch-level [151]	TC	general	GPT-3.5/4, LLaMA2-7B, Mixtral-7B	Solver	sentence	✗
Recommender [234]	TIp	general	GPT-2	Solver	sentence	✗
TabLLM [224]	TIp	general	T0-3B/11B, GPT-3	Solver	sentence	✗
Wadhwa [64]	EM	general	Flan-T5, Mistral-7B, Alpaca	Matcher, Expert	attr-value pairs	✗
Steiner [65]	EM	general	LLaMA3.1-8B/70B, GPT-4o(-mini)	Matcher, Expert	only values	✗
Self-Refinement [31]	CTA	general	LLaMA-8B/70B, GPT-4o(-mini)	Solver, Expert	markdown	✗
Table-GPT [39]	TC,TIp,CTA,SM,EM	general	GPT-3.5, ChatGPT	Solver	markdown	✗
CoddLLM [236]	CTA	general	Claude-3.5-Sonnet, Mistral-NeMo-12B	Solver, Expert	markdown	✗
TableLlama [41]	CTA,CEA,CPA	general	LLaMA-7B	Solver	row-identifier	✗
Jellyfish [40]	TC,TIp,EM,SM,CTA	general	Mistral-7B, LLaMA3-8B, OpenOrca-Platypus2-13B	Solver, Expert	attr-value pairs	✗
Table-Specialist [111]	TIp,SM,EM,CTA,ST	general	GPT-3.5	Solver, Checker	markdown	✗

the detector's explanation with contextual data. The process iterates until no further errors are detected or a maximum iteration threshold is reached. AutoDCWorkflow [147] takes a step further by framing LLMs as autonomous agents that assess data quality and plan correction strategies aligned with analytical goals. Besides, it enhances robustness through step-by-step reasoning and constrains LLM-generated operations to a predefined action pool, improving reliability.

4) LLM as Agent: Data cleaning is a critical processes that prepare raw data for analysis, ensuring high quality and reliability. To this end, given a specific data analysis purpose, AutoDCWorkflow [147] presents an LLM-based pipeline that automatically generating data-cleaning workflows. Concretely, it leverages LLMs to translate user intents into a sequence of executable OpenRefine operations to produce a minimal, clean table. However, this framework still focus on single table tasks, and the interpretability and debuggability of the generated workflows remain constrained.

Discussion: Integrating LLMs into table correction pipelines significantly improves flexibility and contextual reasoning. However, the repair stage often requires a degree of commonsense and domain-specific knowledge that may exceed the capabilities of even advanced LLMs, leading to hallucinations or inaccurate suggestions. While retrieval-augmented frameworks

[2]https://huggingface.co/upstage/SOLAR-0-70b-16bit

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

address this to some extent, their reliance on user-provided data lakes limits scalability. Furthermore, the open-ended nature of LLM-generated corrections underscores the need for reliable verification mechanisms, which remain an open challenge in practice.

B. Table Imputation

Traditional statistical imputation methods are theoretically sound but often lack semantic understanding. In contrast, modern machine learning approaches, based on matrix factorization and deep learning, are capable of capturing superficial patterns and relationships within the data. However, selecting an optimal imputation method remains challenging, as it requires careful consideration of various factors, including data types, the nature of missingness, and the proportion of missing values. Recently, LLMs have emerged as a promising alternative for table imputation, due to their ability to perform commonsense reasoning and model complex interdependencies. Existing LLM-based imputation methods can be broadly categorized based on the role LLMs play: LLM as Expert and LLM as Solver.

1) LLM as Expert: In this paradigm, LLMs are employed as sources of external knowledge to augment traditional or LLM-based imputation frameworks [134], [136], [137]. For instance, MonotonicLR [136] leverages LLMs to infer inter-column relationships and reorder values in correlated columns accordingly. CLAIM [137] emphasizes the use of contextual information by generating natural language descriptions of missing data and fine-tuning LLMs with a contextualized dataset to improve imputation accuracy. Similarly, Ji et al. [134] use an LLM to generate relevant questions that guide demonstration selection for more effective prediction.

2) LLM as Solver: In this setting, LLMs are prompted to directly impute missing values through carefully crafted prompts [72], [133], [134], [135]. The performance of such approaches depends heavily on the quality of demonstrations and the construction of informative context. Unlike earlier work [72], which selects demonstrations manually, more recent methods [75], [133] dynamically retrieve examples based on similarity metrics. To enrich the input context, Ji et al. [134] employ GPT-4-turbo to synthesize responses from the top-k retrieved question-answer pairs, while UniDM [67] introduces a context parsing module to convert raw inputs into more interpretable forms for LLMs.

To address the issue of computational efficiency, FS [74] applies advanced prompt engineering techniques such as contextualization and feature selection. Additionally, Akella et al. [75] propose a reinforcement learning-based algorithm to dynamically identify and sequence task-relevant columns. Moving beyond prompt engineering, LLM-Forest [135] introduces an ensemble-based method inspired by random forests, in which a "forest" of few-shot LLM predictors (trees) is aggregated using confidence-weighted voting to enhance both accuracy and stability.

Discussion: As with table correction, real-world imputation often requires access to external knowledge sources that exceed the information embedded in an LLM's pre-trained parameters. Thus, providing sufficient, high-quality contextual information is essential. Furthermore, concerns regarding data security and privacy must be addressed when integrating external data. Lastly, relying solely on LLMs introduces challenges related to (1) hallucinated or semantically inconsistent outputs, and (2) the inherently open-ended nature of LLM completions, underscoring the need for robust validation mechanisms in practical deployments.

C. Table Normalization

1) Semantic Canonicalization: For *semantic canonicalization*, LLMs play three main roles: Expert, Matcher and Selector. The latter two both function as solvers, differing only in specific problem formulations.

a) LLM as Expert: LLMs can serve as semantic experts by leveraging their internal knowledge to identify synonyms and generate descriptive definitions [62]. For example, EDC [62] uses LLMs to generate definitions for entities and relations, which are then encoded as vectors to retrieve semantically similar candidates via vector similarity search.

b) LLM as Matcher: Beyond providing contextual information, LLMs can act as semantic matchers, determining whether two elements refer to the same concept [62], [113], [114]. For instance, EDC [62] prompts the LLM with contextualized queries such as "Can the relation 'participatedIn' be replaced by 'mission' given the current context?" to judge semantic equivalence. To reduce the number of LLM calls, clustering-based methods [113] apply traditional clustering algorithms (e.g., hierarchical agglomerative clustering) and use LLMs to correct low-confidence assignments by reassigning mismatched points. In contrast, Probing [114] avoids prompting by analyzing whether LLM-generated embeddings capture linguistic patterns useful for synonym detection using a K-means-based clustering approach.

c) LLM as Selector: One-to-one pairwise matching can be computationally expensive and may overlook global context, resulting in suboptimal outcomes. To address this, some approaches prompt LLMs to act as selectors, identifying synonym pairs from a candidate set [59], [112]. For example, WDC-PAVE [112] employs prompt templates to extract and normalize attribute-value pairs from product data. AutoAlign [59] constructs a predicate-proximity graph to capture synonym relationships between predicates, with LLMs identifying synonymous entity-type pairs through prompt engineering.

2) Syntactic Transformation: Traditional approaches to syntactic transformation, such as Pandas [238] and TDE [118], rely heavily on extensive code libraries tailored to specific data types and formats. While these tools are widely used, they are not particularly user-friendly, often requiring users to manually identify and apply appropriate functions. Additionally, their coverage of transformation patterns is limited. In contrast, LLMs offer a promising alternative, as they combine domain knowledge and code generation capabilities to handle a broader range of transformation tasks with greater adaptability. Current LLM-based transformation methods can be categorized into three roles: LLM as Solver, LLM as Coder, and LLM as Agent.

a) LLM as Solver: In this setting, LLMs perform data transformations directly through text-based outputs. The effectiveness of such methods is highly dependent on prompt design, partic-

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

ularly the clarity of transformation instructions and the quality of demonstration examples. For instance, Narayan et al. [72] show that GPT-3 (175B) achieves significant improvements over zero-shot settings when provided with curated demonstrations, outperforming the state-of-the-art system TDE [118]. Similarly, Brinkmann et al. [112] use GPT-3.5 and GPT-4 to normalize attribute values from product titles and descriptions using different prompt templates. Their findings indicate that demonstrations have minimal impact on GPT-3.5 but improve GPT-4's F1-score by 4%. UniDM [67] further observes that cloze-style prompts, which are written in natural language with placeholders, are especially effective for guiding LLMs in transformation tasks. In domain-specific scenarios, such as the Internet of Things (IoT), IDSM+TRGM [109] fine-tunes LLMs to convert heterogeneous sensor data into standardized formats, highlighting the adaptability of LLMs for specialized transformation tasks.

b) LLM as Coder: Rather than directly outputting transformed data, LLMs can also generate transformation scripts or rules, enhancing transparency, scalability, and reusability. For example, IDSM+TRGM [109] uses GPT-4 to generate scripts for standardizing sensor data, and fine-tunes the model on a curated dataset of 100 complex transformation examples. SQL-Morpher [108] introduces an iterative prompting framework that integrates flaw detection and schema-aware hints to guide SQL-based transformations. DATAMORPHER [110] enhances token efficiency by profiling the input table to extract salient features and using historical examples to guide code generation, while also employing ranking-based feature selection to reduce prompt noise and cost. NormTab [107] improves efficiency by asking the LLM to first identify which columns require normalization, splitting the table into normalized and unnormalized parts, and applying transformations only to the latter. Additionally, FSTO-Gen [37] employs LLMs to generate Functional Semantic Types (FSTs) along with normalization, validation, and casting code, enabling efficient automation of human-intensive data tasks in collaborative environments.

c) LLM as Agent: While Solver- and Coder-based methods demonstrate LLMs' abilities in direct transformation and code generation, they often require technical expertise. To reduce the barrier for non-expert users, Agent-based methods utilize LLMs as intermediaries that orchestrate the transformation process by selecting appropriate operations based on the context. Clean-Agent [106] exemplifies this approach through a declarative Python library that automates data transformation using a four-module pipeline: (1) a Chat Manager that retains conversational history, (2) a Column-Type Annotator that uses an LLM to determine semantic column types, (3) a Python Programmer that generates executable transformation code based on annotations and prior dialogue, and (4) a Code Executor that runs the code and updates the data accordingly.

Discussion: LLM-based normalization methods provide a powerful alternative to traditional systems, particularly in low-resource or ad hoc settings. By leveraging textual prompts and a few demonstration examples, LLMs can quickly perform normalization tasks, significantly reducing the technical barrier for non-expert users. However, current approaches predominantly depend on closed-source models such as GPT-3.5 and GPT-4, raising critical concerns regarding cost, scalability, and data privacy. Furthermore, the effectiveness of these methods remains highly sensitive to prompt design, which can lead to inconsistent performance across tasks and domains.

D. Table Integration

Early table integration methods leveraging pre-trained language models (PLMs) such as BERT and RoBERTa suffer from two primary limitations: (1) a strong dependence on task-specific labeled data, and (2) limited generalization to out-of-distribution elements. Large language models (LLMs) offer a promising alternative by reducing reliance on annotated data and improving generalization across domains. In the context of table integration, LLMs can assume a variety of roles, including Matcher, Selector, Expert, and Agent, each corresponding to a different task formulation or function. While both Matcher and Selector act as the Solver, they differ in the structure of input and the strategy for decision-making.

1) LLM as Matcher: In the matcher role, LLMs directly evaluate whether two records refer to the same real-world entity using queries such as "Do these two records refer to the same real-world entity?" [58]. Existing methods mainly fall into two categories: prompt engineering and fine-tuning. (a) Prompt-based methods typically rely on carefully designed queries and in-context learning (ICL). For instance, UniDM [67] first assess the relevance between a target and candidate record, then serializes the context into text for LLM reasoning. Other studies [63], [71] explore different demonstration selection strategies, such as random, similarity-based, and curated examples, showing that performance is highly sensitive to example quality. Prompt design also matters: filtering noisy attributes, choosing relevant ones, and even subtle changes in wording can significantly affect results [72]. To reduce the high cost of ICL, BatChER [66] proposes a batch prompting strategy, while Li et al. [68] improve efficiency by verifying only the most informative record pairs. Some methods further enhance reasoning with CoT prompting [73], [74] or retrieval-based knowledge augmentation. For instance, KCMF [70] incorporates pseudo-code decomposition and dataset-derived knowledge, while KG-RAG4SM [69] retrieves relevant knowledge subgraphs to support semantic matching. (b) Fine-tuning-based methods adapt LLMs to EM using task-specific supervision. Existing studies show that fine-tuned models can outperform stronger closed-source LLMs in some settings [63], and that distilling CoT-style reasoning from larger models can further improve the performance and generalization of smaller ones [64]. However, fine-tuning may weaken cross-domain transfer, especially for larger models [65].

2) LLM as Selector: Exhaustive pairwise comparison requires $n(n-1)/2$ comparisons for n records, impractical at scale and lacking global context. Instead, selector-based methods [59], [61] reframe the task with queries such as "Select the record from this list that refers to the same real-world entity as the given record". COMEM [58] compares matching, selecting, and a hybrid compare paradigm. It finds that (1) selection is often most cost-effective for fine-grained identification due to its global context awareness, (2) a filtering-then-identifying

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

pipeline enhances performance, and (3) no single LLM dominates across all strategies. GRAM [61] proposes a hierarchical multi-class classification framework, where an LLM-based classifier operates at each non-leaf node to limit and refine candidate sets. It also introduces dynamic prompt selection based on input features to improve both speed and accuracy.

3) LLM as Expert: Given the cost of real-time LLM inference, some methods distill LLM reasoning into smaller, more efficient models. For example, Wadhwa et al. [64] extract CoT-style explanations from LLMs to fine-tune small models capable of generating both match decisions and justifications. Few-shot prompting is used to elicit more generalizable reasoning chains. Steiner et al. [65] enhance training sets with LLM-generated explanations and use error-based sample selection, focusing fine-tuning on examples likely to be misclassified. Other work explores rule extraction. For instance, Peeters et al. [63] use LLMs to generate matching rules from examples. However, they report that rule-guided GPT-4 underperforms its zero-shot baseline by 1–3% in F1-score, while open models like LLaMA2-70B, LLaMA3.1-70B, and Mixtral-8× 7B show gains of up to 17% in F1 across datasets.

4) LLM as Agent: Real-world table integration often involves multiple interdependent subtasks (e.g., schema matching, value matching, and data transformation), making the integration pipeline complex and fragmented. Harmonia [60] addresses this with an interactive system that combines LLM reasoning, a user interface, and a library of data integration tools. In this setting, LLMs serve dual roles: (1) communicators, interacting with users to ensure process correctness, and (2) tool callers or code generators when existing algorithms fall short. This agentic system streamlines and automates end-to-end integration workflows. Dango [239] is a mixed-initiative system designed to enhance the communication between LLM agents and usersin data wrangling workflows. It supports both direct demonstration and NL interaction, enabling users to express their intent more flexibly and effectively. To address the complexity of multi-table tasks, Dango also provides automatic data provenance visualization, which helps users understand the dependencies and relationships across tables.

Discussion: Integrating tables from diverse real-world sources is a complex and multi-faceted challenge. Beyond core tasks like schema and entity matching, it also involves a range of auxiliary processes such as normalization and semantic interpretation. While systems like Harmonia [60] demonstrate the potential of LLMs in this space, their role remains largely assistive, supporting users rather than operating independently. Users are still required to guide the workflow and validate outcomes. To fully unlock the capabilities of LLMs in table integration, there is a pressing need for more autonomous and efficient agent-based systems that can coordinate and execute end-to-end workflows with minimal human intervention.

E. Table Interpretation

Traditional table interpretation methods face two major limitations: (1) a lack of sufficient contextual information to resolve semantic ambiguities, and (2) a scarcity of labeled data tailored to specific domains or tasks. LLMs, with their strong capabilities in natural language understanding and reasoning, offer a promising solution to these challenges. Recent research has increasingly explored LLM-based approaches to table interpretation. These methods can be broadly categorized based on the role played by the LLM: LLM as Expert, LLM as Solver, and LLM as Coder.

1) LLM as Expert: Contextual information surrounding a table, such as content from captions, footnotes, and surrounding text, can provide rich semantics essential for interpretation. However, extracting and integrating such context remains a challenge. To address this, Oshima et al. [42] propose a rule-based method to extract global document context, which is then refined by an LLM. The LLM enriches this context by injecting supplemental background knowledge and clarifying ambiguous references. Another study by Korini et al. [31] explores self-refined term definitions based on validation feedback. While this hybrid method shows promise, it slightly underperforms compared to task-specific fine-tuning, highlighting the complexity of balancing LLM-generated knowledge with learned model behavior.

2) LLM as Solver: Several studies use LLMs to directly perform table interpretation tasks [32], [34], [35], [36], [37]. For instance, Korini et al. [33] develop zero-shot prompts for column property annotation (CPA) and observe that while GPT-3.5 is highly sensitive to prompt design, GPT-4 delivers more stable and superior performance. Moreover, fine-tuned GPT-3.5 surpasses a RoBERTa-based model trained on the same dataset by 11% F1. In follow-up work [32], three prompt styles (i.e., column-based, text-based, and table-based) are compared, with table-based prompts consistently outperforming the others in both zero- and few-shot settings. Similarly, ArcheType [36] evaluates six different serialization strategies and dynamically selects among them at inference time based on context. To enhance interpretability and structure of the results, several methods [32], [35] adopt step-by-step instructional prompts. For example: "1. Format the input as a table. 2. Examine cell values. 3. For each column, determine its relationship to Column 1. 4. Use the format 'Column 2: relationship'. 5. Use only labels from the provided set." Recognizing inter-task dependencies (e.g., CTA supporting CPA), CHORUS [34] introduces a multi-turn dialogue framework capable of flexibly handling various interpretation tasks. To mitigate hallucination propagation, referred to as hallucination snowballing [240], CHORUS incorporates an anchoring mechanism that halts the dialogue upon detecting critical inconsistencies.

Despite their capabilities, LLMs are limited by their static knowledge. To address this, RAG techniques are being applied. For example, RACOON [38] retrieves entity labels and one-hop neighbors from a knowledge graph, refines them using an LLM, and integrates them into the prompt, significantly boosting task accuracy.

3) LLM as Coder: An alternative perspective involves reframing interpretation as a code generation problem. GenSTI [30] finds that structured representations in code form convey more semantic information than free-text descriptions. It leverages Code-LLMs to jointly perform target de-

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

tection and annotation, exploiting interdependencies between sub-tasks.

Discussion: LLMs demonstrate impressive generalization and reasoning abilities across table interpretation tasks. However, most existing methods still rely heavily on prompt engineering to adapt these models, limiting scalability and robustness. Furthermore, while some efforts recognize synergies between sub-tasks (e.g., CPA and CTA), current systems remain largely pipeline-based, restricting their ability to model rich task interdependencies dynamically. Future work should explore multi-task, unified architectures that can interpret tables more holistically, with stronger reasoning consistency and interaction integrity.

VI. A Comparative Analysis Between LLM-Based and LLM-Free Methods

Section V presented a comprehensive overview of current LLM-based methods for tabular data wrangling, where LLMs refer to models with over 1 billion parameters. For comparison, Section II-B summarized contemporary LLM-free approaches. Compared with LLM-free models, which fall short in handling complex patterns or out-of-distribution data, LLMs introduce several distinctive advantages through the whole tabular data wrangling process:

- *Unified interface across heterogeneous tasks:* LLM-free methods are typically designed for specific tasks, requiring separate pipelines and task-specific feature engineering. In contrast, LLMs provides a unified interface that can handle diverse wrangling tasks within a single framework. This significantly enables cross-task generalization. Besides, LLMs support natural language instructions and interactive refinement, making it particularly suitable for real-world, human-in-loop scenarios.
- *Reduced reliance on structured supervision:* LLM-free approaches often depend on labeled datasets or handcrafted rules. LLMs, leveraging large-scale pretraining, can perform many wrangling tasks in zero-shot or few-shot settings, substantially lowering the cost of annotation and manual feature design, particularly in low-resource or domain-transfer scenarios.
- *Enhanced capability for open-world alignment:* LLM-free approaches often struggle with semantic variability and unseen values, while LLMs, embedded with extensive world knowledge, can be better handle such open-world scenarios, including long-tail or previously unseen objects or tasks.

To assess their practical effectiveness, we compare both paradigms across several widely used benchmarks (Fig. 7). To ensure a fair and consistent comparison, we select one or two representative and widely used benchmark datasets for each task, and compile the reported results from the corresponding papers. The performance is uniformly measured using the F1 score. For example, the results for the imputation (TIp) task are collected from [234]. Our key observations are as follows:

- LLM-based methods generally outperform LLM-free methods across a broad range of tasks, though this advantage often comes with significantly higher computational costs.
- Smaller LLMs (i.e., those with fewer than 10 billion parameters), even after fine-tuning, often struggle to perform deep reasoning required for complex tabular tasks.
- GPT-family models (e.g., GPT-3.5, GPT-4) consistently achieve stronger performance compared to other LLM architectures, especially in zero- and few-shot settings.
- LLM-free methods still maintain an edge in certain scenarios, particularly those requiring symbolic reasoning, linguistic probing, or rule-based matching [114], [234].

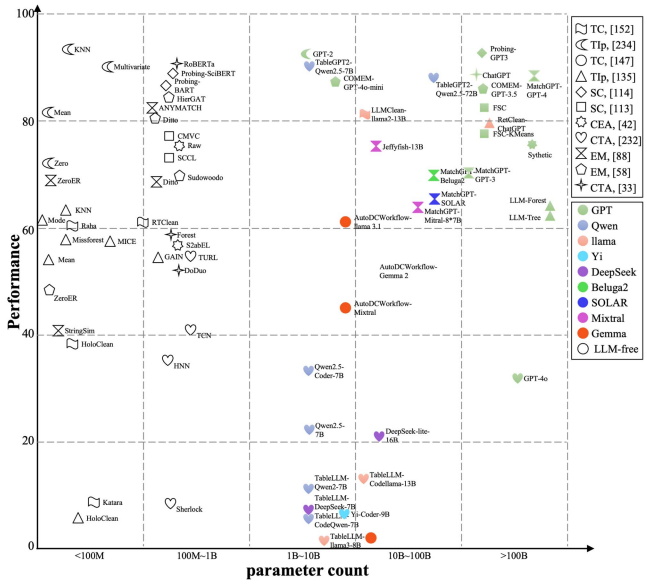


Fig. 7. A Comparison of LLM-based and LLM-free methods in terms of parameter count and model performance, where colored icons denote LLM-based methods and uncolored icons denote LLM-free methods.

These findings suggest that neither approach is universally superior, and that hybrid strategies combining the strengths of both paradigms may offer the most promising path forward. Existing hybrid systems typically follow one of two designs: (1) using LLMs to verify or refine the outputs of LLM-free models, or (2) using LLMs to inject contextual signals or background knowledge to guide traditional pipelines. However, these designs often treat LLMs and LLM-free methods as two independent modules. Future research should explore tightly integrated hybrid frameworks that enable dynamic collaboration between LLMs and traditional systems, possibly through feedback loops, or joint training strategies. Such frameworks may unlock new capabilities in table understanding, particularly in domains with strict efficiency or interpretability.

VII. Conclusion and Future Direction

The survey is the first a comprehensive investigation into leveraging large language models (LLMs) for tabular data wrangling, encompassing table correction, imputation, integration, etc. We first categorized tabular data wrangling tasks based on the type of "dirty data". We then provide a thorough review,

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

analysis and comparison of methods in each task. As discussed in Section V and VI, LLMs have shown remarkable potential across a wide range of tabular data wrangling tasks. Their flexibility, contextual understanding, and ability to unify multiple sub-tasks under a common framework distinguish them from traditional methods. However, despite their capabilities, LLMs face a number of persistent challenges, driven by both intrinsic model limitations and the structural complexity of real-world tabular data. Below, we outline several key areas for future research:

1) Domain Knowledge: LLMs are primarily trained on general web-scale corpora, and often lack the specialized knowledge required to handle domain-specific tables containing uncommon symbols, units, or terminology. While techniques such as continual pre-training and task-specific fine-tuning can partially bridge this gap, they introduce risks such as catastrophic forgetting [241]. RAG offers a more dynamic solution by incorporating external knowledge on demand, but its effectiveness depends heavily on retrieval quality, irrelevant or noisy documents can degrade overall performance.

2) Numerical Reasoning: LLMs frequently underperform on tasks involving numerical reasoning due to their reliance on subword tokenization [242], which is ill-suited for numeric values. Given the prevalence of numerical data in tables, future work should explore numerically-aware tokenization schemes, hybrid symbolic-neural approaches, or the integration of external arithmetic modules to enhance numerical fidelity.

3) Categorical Value Understanding: Tabular data often includes categorical fields with implicit, context-dependent meanings, for instance, "1" might denote "yes", "unknown", or a class label depending on the column. The same value may convey different semantics across columns or datasets. Future methods should include dedicated categorical disambiguation mechanisms that improve LLM robustness in interpretation and cleaning tasks.

4) Agent-based Wrangling: Tabular data wrangling typically requires multi-step workflows composed of interdependent operations. Recent work hints at the potential of LLM-powered agents capable of planning, executing, verifying, and iterating over complex tasks. Future frameworks should aim to integrate these abilities into modular agent-based systems, enabling dynamic orchestration of sub-modules and external tools in a closed feedback loop.

5) Multi-Modal Reasoning: Tables embedded in scanned documents, reports, or images often encode visual or layout-specific cues (e.g., color, alignment, headers) that are absent from raw text formats. While multi-modal LLMs (MLLMs) [232], [243] have shown success in table question answering and generation tasks, their utility for wrangling tasks remains largely untapped.

6) Token Limitations: Real-world tabular datasets are often characterized by large numbers of rows and columns, while the token-length constraints of LLMs (e.g., 16K–128 K tokens) pose significant challenges when processing these tables. Table serialization, especially for complex tables, can inflate token usage and compromise structure preservation. Developing scalable architectures and efficient table representation techniques that preserve global context remains an important direction for future research.

7) Cross-Table Reasoning: Many real-world table-related tasks, such as error detection and correction, require reasoning across multiple tables rather than within a single table. However, most existing LLM-based methods operate on isolated tables and lack mechanisms for modeling inter-table dependencies. Future work should explore methods for cross-table representation learning and reasoning, enabling LLMs to capture relationships across heterogeneous data sources.

8) Computational Cost: Most existing studies evaluate LLM-based methods on relatively small tables, where these methods have shown strong performance and generalization across different table-related tasks. Nevertheless, their high token consumption results in considerable computational and economical costs, which hinders their deployment in real-world systems. Future research should focus on lightweight architectures, efficient prompting strategies, and hybrid pipelines that combine LLMs with traditional methods to achieve a better trade-off between performance and efficiency.

REFERENCES

[1] S. Mohammed et al., "The effects of data quality on machine learning performance," 2022, *arXiv:2207.14529*.

[2] N. Shukla, M. Sankar, and S. Palani, *Data Wrangling on AWS: Clean and Organize Complex Data for Analysis*. Birmingham, U.K.: Packt Publishing Ltd, 2023.

[3] S. Probstein, "Reality check: Still spending more time gathering instead of analyzing," Forbes Technology Council, Dec. 17, 2019.

[4] DB-Engines, "Ranking scores per category in percent," Aug. 2024.

[5] W. Fan, F. Geerts, and X. Jia, "Improving data quality: Consistency and accuracy," in *Proc. 33rd Int. Conf. Very Large Data Bases*, 2007, pp. 315–326.

[6] L. Galárraga, G. Heitz, K. Murphy, and F. M. Suchanek, "Canonicalizing open knowledge bases," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2014, pp. 1679–1688.

[7] T. Rekatsinas, X. Chu, I. F. Ilyas, and C. Ré, "HoloClean: Holistic data repairs with probabilistic inference," 2017, *arXiv:1702.00820*.

[8] M. Dallachiesa et al., "Nadeef: A commodity data cleaning system," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2013, pp. 541–552.

[9] S. Kandel, A. Paepcke, J. Hellerstein, and J. Heer, "Wrangler: Interactive visual specification of data transformation scripts," in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, 2011, pp. 3363–3372.

[10] M. Hulsebos et al., "Sherlock: A deep learning approach to semantic data type detection," in *Proc. ACM Int. Conf. Knowl. Discov. Data Mining*, 2019, pp. 1500–1508.

[11] Y. Zhou, S. Singh, and C. Christodoulopoulos, "Tabular data concept type detection using star-transformers," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2021, pp. 3677–3681.

[12] J. Herzig, P. K. Nowak, T. Mueller, F. Piccinno, and J. Eisenschlos, "TAPAS: Weakly supervised table parsing via pre-training," in *Proc. 58th Annu. Meet. Assoc. Comput. Linguist.*, 2020, pp. 4320–4333.

[13] P. Yin, G. Neubig, W.-T. Yih, and S. Riedel, "TaBERT: Pretraining for joint understanding of textual and tabular data," in *Proc. Conf. Assoc. Comput. Linguist.*, 2020, pp. 8413–8426.

[14] H. Iida, D. Thai, V. Manjunatha, and M. Iyyer, "TABBIE: Pretrained representations of tabular data," in *Proc. Conf. Assoc. Comput. Linguist.*, 2021, pp. 3446–3456.

[15] A. Radford et al., "Language models are unsupervised multitask learners," *OpenAI Blog*, vol. 1, no. 8, 2019, Art. no. 9.

[16] J. Achiam et al., "GPT-4 technical report," 2023, *arXiv:2303.08774*.

[17] H. Touvron et al., "Llama: Open and efficient foundation language models," 2023, *arXiv:2302.13971*.

[18] A. Chowdhery et al., "PaLM: Scaling language modeling with pathways," *J. Mach. Learn. Res.*, vol. 24, no. 240, pp. 1–113, 2023.

[19] J. Bai et al., "Qwen technical report," 2023, *arXiv:2309.16609*.

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

[20] J. Wei et al., "Chain-of-thought prompting elicits reasoning in large language models," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 24824–24837.
[21] Y. Liu, "RoBERTa: A robustly optimized bert pretraining approach," 2019, *arXiv:1907.11692*.
[22] W. Lu, J. Zhang, J. Fan, Z. Fu, Y. Chen, and X. Du, "Large language model for table processing: A survey," *Front. Comput. Sci.*, vol. 19, no. 2, 2025, Art. no. 192350.
[23] X. Fang et al., "Large language models (LLMs) on tabular data: Prediction, generation, and understanding–A survey," 2024, *arXiv:2402.17944*.
[24] H. Dong and Z. Wang, "Large language models for tabular data: Progresses and future directions," in *Proc. Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2024, pp. 2997–3000.
[25] M. Cheng et al., "A survey on table mining with large language models: Challenges, advancements and prospects," *Authorea Preprints*, 2025.
[26] Y. Ruan, X. Lan, J. Ma, Y. Dong, K. He, and M. Feng, "Language modeling on tabular data: A survey of foundations, techniques and evolution," 2024, *arXiv:2408.10548*.
[27] X. Wu, A. Ritter, and W. Xu, "Tabular data understanding with LLMs: A survey of recent advances and challenges," 2025, *arXiv:2508.00217*.
[28] S. Raieli, "Tabular deep learning: A survey from small neural networks to large language models," *Authorea Preprints*, 2025.
[29] X. Zhang, D. Wang, L. Dou, Q. Zhu, and W. Che, "A survey of table reasoning with large language models," *Front. Comput. Sci.*, vol. 19, no. 9, 2025, Art. no. 199348.
[30] V.-P. Huynh, Y. Chabot, and R. Troncy, "Towards generative semantic table interpretation," in *Proc. VLDB Workshops*, 2023, pp. 26565–26577.
[31] K. Korini and C. Bizer, "Evaluating knowledge generation and self-refinement strategies for LLM-based column type annotation," 2025, *arXiv:2503.02718*.
[32] K. Korini and C. Bizer, "Column type annotation using ChatGPT," 2023, *arXiv:2306.00745*.
[33] K. Korini and C. Bizer, "Column property annotation using large language models," in *Proc. Eur. Semantic Web Conf.*, Springer, 2024, pp. 61–70.
[34] M. Kayali, A. Lykov, I. Fountalis, N. Vasiloglou, D. Olteanu, and D. Suciu, "Chorus: Foundation models for unified data discovery and exploration," 2023, *arXiv:2306.09610*.
[35] A. Babamahmoudi, "Column type annotation using large language models," 2024.
[36] B. Feuer, Y. Liu, C. Hegde, and J. Freire, "Archetype: A novel framework for open-source column type annotation using large language models," 2023, *arXiv:2310.18208*.
[37] S. Konan, L. Rudolph, and S. Affens, "Automating the generation of a functional semantic types ontology with foundational models," in *Proc. Conf. North Am Chap. Assoc. Comput. Linguist.: Hum. Lang. Technol.*, 2024, pp. 248–265.
[38] L. L. Wei, G. Xiao, and M. Balazinska, "RACOON: An LLM-based framework for retrieval-augmented column type annotation with a knowledge graph," 2024, *arXiv:2409.14556*.
[39] P. Li et al., "Table-GPT: Table-tuned GPT for diverse table tasks," 2023, *arXiv:2310.09263*.
[40] H. Zhang, Y. Dong, C. Xiao, and M. Oyamada, "Jellyfish: A large language model for data preprocessing," 2023, *arXiv:2312.01678*.
[41] T. Zhang, X. Yue, Y. Li, and H. Sun, "TableLlama: Towards open large generalist models for tables," 2023, *arXiv:2311.09206*.
[42] Y. Oshima, H. Shindo, H. Teranishi, H. Ouchi, and T. Watanabe, "Synthetic context with LLM for entity linking from scientific tables," in *Proc. 4th Workshop Scholarly Document Process.*, 2024, pp. 202–214.
[43] Y. Wang, H. Xin, and L. Chen, "KGLink: A column type annotation method that combines knowledge graph and pre-trained language model," 2024, *arXiv:2406.00318*.
[44] N. Abdelmageed and S. Schindler, "JenTab: Matching tabular data to knowledge graphs," in *Proc. Semantic Web Challenge Tabular Data Knowl. Graph Matching, Int. Semantic Web Conf.*, 2020, pp. 40–49.
[45] J. Chen, E. Jiménez-Ruiz, I. Horrocks, and C. Sutton, "ColNet: Embedding the semantics of web tables for column type prediction," in *Proc. AAAI Conf. Artif. Intell.*, 2019, pp. 29–36.
[46] U. Khurana and S. Galhotra, "Semantic concept annotation for tabular data," in *Proc. ACM Int. Conf. Inf. Knowl. Manage.*, 2021, pp. 844–853.
[47] D. Oliveira and M. d'Aquin, "ADOG-annotating data with ontologies and graphs," in *Proc. Semantic Web Challenge Tabular Data Knowl. Graph Matching, Int. Semantic Web Conf.*, 2019, pp. 1–6.
[48] X. Deng, H. Sun, A. Lees, Y. Wu, and C. Yu, "TURL: Table understanding through representation learning," *ACM SIGMOD Rec.*, vol. 51, no. 1, pp. 33–40, 2022.
[49] S. K. Ramnandan, A. Mittal, C. A. Knoblock, and P. Szekely, "Assigning semantic labels to data sources," in *Proc. Eur. Semantic Web Conf.*, 2015, pp. 403–417.
[50] M. Pham, S. Alse, C. A. Knoblock, and P. Szekely, "Semantic labeling: A domain-independent approach," in *Proc. Semantic Web Challenge Tabular Data Knowl. Graph Matching, Int. Semantic Web Conf.*, 2016, pp. 446–462.
[51] D. Zhang, M. Hulsebos, Y. Suhara, Ç. Demiralp, J. Li, and W.-C. Tan, "Sato: Contextual semantic type detection in tables," in *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 1835–1848, 2020.
[52] D. Wang, P. Shiralkar, C. Lockard, B. Huang, X. L. Dong, and M. Jiang, "TCN: Table convolutional network for web table interpretation," in *Proc. Web Conf.*, 2021, pp. 4020–4032.
[53] Z. Miao and J. Wang, "Watchog: A light-weight contrastive learning based framework for column annotation," in *Proc. ACM Manag. Data*, vol. 1, no. 4, pp. 1–24, 2023.
[54] X. Su and Y. Shen, "Tab-HGNN: Learning column representation with heterogeneous graph neural network for web table interpretation," in *Proc. IEEE Int. Conf. Big Data*, 2022, pp. 1142–1151.
[55] J. Liu, V.-P. Huynh, Y. Chabot, and R. Troncy, "Radar station: Using KG embeddings for semantic table interpretation and entity disambiguation," in *Proc. Int. Semant. Web Conf.*, 2022, pp. 498–515.
[56] Y. Suhara et al., "Annotating columns with pre-trained language models," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2022, pp. 1493–1503.
[57] Y. Li, J. Li, Y. Suhara, A. Doan, and W.-C. Tan, "Deep entity matching with pre-trained language models," 2020, *arXiv:2004.00584*.
[58] T. Wang et al., "Match, compare, or select? An investigation of large language models for entity matching," 2024, *arXiv:2405.16884*.
[59] R. Zhang et al., "AutoAlign: Fully automatic and effective knowledge graph alignment enabled by large language models," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 6, pp. 2357–2371, Jun. 2024.
[60] A. Santos, E. H. Pena, R. Lopez, and J. Freire, "Interactive data harmonization with LLM agents," 2025, *arXiv:2502.07132*.
[61] X. Liu, R. Wang, Y. Song, and L. Kong, "GRAM: Generative retrieval augmented matching of data schemas in the context of data security," in *Proc. ACM Int. Conf. Knowl. Discov. Data Mining*, 2024, pp. 5476–5486.
[62] B. Zhang and H. Soh, "Extract, define, canonicalize: An LLM-based framework for knowledge graph construction," 2024, *arXiv:2404.03868*.
[63] R. Peeters, A. Steiner, and C. Bizer, "Entity matching using large language models," 2023, *arXiv:2310.11244*.
[64] S. Wadhwa, A. Krishnan, R. Wang, B. C. Wallace, and C. Kong, "Learning from natural language explanations for generalizable entity matching," 2024, *arXiv:2406.09330*.
[65] A. Steiner, R. Peeters, and C. Bizer, "Fine-tuning large language models for entity matching," 2024, *arXiv:2409.08185*.
[66] M. Fan et al., "Cost-effective in-context learning for entity resolution: A design space exploration," in *Proc. IEEE Int. Conf. Data Eng.*, IEEE, 2024, pp. 3696–3709.
[67] Y. Qian et al., "UniDM: A unified framework for data manipulation with large language models," in *Proc. Mach. Learn. Syst.*, vol. 6, pp. 465–482, 2024.
[68] H. Li et al., "On leveraging large language models for enhancing entity resolution," 2024, *arXiv:2401.03426*.
[69] C. Ma, S. Chakrabarti, A. Khan, and B. Molnár, "Knowledge graph-based retrieval-augmented generation for schema matching," 2025, *arXiv:2501.08686*.
[70] Y. Xu, H. Li, K. Chen, and L. Shou, "KcMF: A knowledge-compliant framework for schema and entity matching with fine-tuning-free LLMs," 2024, *arXiv:2410.12480*.
[71] A. Yuan, "Role of demonstration for entity matching," Ph.D. dissertation, Brown University Providence, Rhode Island, 2023.
[72] A. Narayan, I. Chami, L. Orr, and C. Ré, "Can foundation models wrangle your data?," in *Proc. VLDB Endowment*, vol. 16, no. 4, pp. 738–746, 2022.
[73] K. Sisaengsuwanchai, N. Nananukul, and M. Kejriwal, "How does prompt engineering affect ChatGPT performance on unsupervised entity resolution?," 2023, *arXiv:2310.06174*.
[74] H. Zhang, Y. Dong, C. Xiao, and M. Oyamada, "Large language models as data preprocessors," 2023, *arXiv:2308.16361*.
[75] A. Akella, A. Manatkar, B. Chavda, and H. Patel, "An automatic prompt generation system for tabular data tasks," 2024, *arXiv:2405.05618*.

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

[76] G. Papadakis, D. Skoutas, E. Thanos, and T. Palpanas, "Blocking and filtering techniques for entity resolution: A survey," *ACM Comput. Surv.*, vol. 53, pp. 1–42, 2020.

[77] J. Mests and M. Tang, "Distributed representations of tuples for entity resolution," in *Proc. VLDB Endowment*, vol. 11, no. 11, pp. 1454–1467, 2018.

[78] S. Thirumuruganathan et al., "Deep learning for blocking in entity matching: A design space exploration," in *Proc. VLDB Endowment*, vol. 14, no. 11, pp. 2459–2472, 2021.

[79] G. Papadakis, M. Fisichella, F. Schoger, G. Mandilaras, N. Augsten, and W. Nejdl, "Benchmarking filtering techniques for entity resolution," in *Proc. IEEE Int. Conf. Data Eng.*, 2023, pp. 653–666.

[80] D. Paulsen, Y. Govind, and A. Doan, "Sparkly: A simple yet surprisingly strong TF/IDF blocker for entity matching," in *Proc. VLDB Endowment*, vol. 16, no. 6, pp. 1507–1519, 2023.

[81] Y. Guo et al., "Camper: An effective framework for privacy-aware deep entity resolution," in *Proc. ACM Int. Conf. Knowl. Discov. Data Mining*, 2023, pp. 626–637.

[82] M. Ebraheem, S. Thirumuruganathan, S. Joty, M. Ouzzani, and N. Tang, "Deeper–deep entity resolution," 2017, *arXiv:1710.00597*.

[83] J. Kasai, K. Qian, S. Gurajada, Y. Li, and L. Popa, "Low-resource deep entity resolution with transfer and active learning," 2019, *arXiv:1906.08042*.

[84] N. Kirielle, P. Christen, and T. Ranbaduge, "Unsupervised graph-based entity resolution for complex entities," *ACM Trans. Knowl. Discov. Data*, vol. 17, no. 1, pp. 1–30, 2023.

[85] R. Wang, Y. Li, and J. Wang, "Sudowoodo: Contrastive self-supervised learning for multi-purpose data integration and preparation," in *Proc. IEEE Int. Conf. Data Eng.*, 2023, pp. 1502–1515.

[86] J. Wang, Y. Li, W. Hirota, and E. Kandogan, "Machop: An end-to-end generalized entity matching framework," in *Proc. 5th Int. Work. Exploit. Artif. Intell. Technol. Data Manag.*, 2022, pp. 1–10.

[87] L. Fang, L. Li, Y. Liu, V. I. Torvik, and B. Ludäscher, "KAER: A knowledge augmented pre-trained language model for entity resolution," 2023, *arXiv:2301.04770*.

[88] Z. Zhang, P. Groth, I. Calixto, and S. Schelter, "Anymatch–efficient zero-shot entity matching with a small language model," 2024, *arXiv:2409.04073*.

[89] O. Mehdi, H. Ibrahim, and L. Affendey, "An approach for instance based schema matching with google similarity and regular expression," *Int. Arab J. Inf. Technol.*, vol. 14, no. 5, pp. 755–763, 2017.

[90] C. Chen, B. Golshan, A. Y. Halevy, W.-C. Tan, and A. Doan, "BigGorilla: An open-source ecosystem for data preparation and integration," *IEEE Data Eng. Bull.*, vol. 41, no. 2, pp. 10–22, 2018.

[91] M. Zhang, M. Hadjieleftheriou, B. C. Ooi, C. M. Procopiuc, and D. Srivastava, "Automatic discovery of attributes in relational databases," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2011, pp. 109–120.

[92] R. Cappuzzo, P. Papotti, and S. Thirumuruganathan, "Creating embeddings of heterogeneous relational datasets for data integration tasks," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2020, pp. 1335–1349.

[93] Y. Zhang et al., "Schema matching using pre-trained language models," in *Proc. IEEE Int. Conf. Data Eng.*, 2023, pp. 1558–1571.

[94] D. Mukherjee, A. Bandyopadhyay, R. Chowdhury, and I. Bhattacharya, "Learning knowledge graph for target-driven schema matching," in *Proc. 3rd ACM India Joint Int. Conf. Data Sci. Manage. Data*, 2021, pp. 65–73.

[95] N. Alboukaey and A. Joukhadar, "Ontology matching as regression problem," *J. Digit. Inf. Manag.*, vol. 16, no. 1, pp. 33–42, 2018.

[96] L. Bulygin and S. A. Stupnikov, "Applying of machine learning techniques to combine string-based, language-based and structure-based similarity measures for ontology matching," in *Proc. Int. Conf. Data Analyt. Manage. Data Intensive Domains*, 2019, pp. 129–147.

[97] J. Zhang, B. Shin, J. D. Choi, and J. C. Ho, "SMAT: An attention-based deep learning solution to the automation of schema matching," *Adv. Databases Inf. Syst.*, vol. 12843, pp. 260–274, 2021.

[98] R. Shraga and A. Gal, "PowareMatch: A quality-aware deep learning approach to improve human schema matching," *ACM J. Data Inf. Qual.*, vol. 14, no. 3, pp. 1–27, 2022.

[99] R. Shraga, A. Gal, and H. Roitman, "ADnEV: Cross-domain schema matching using deep similarity matrix adjustment and evaluation," in *Proc. VLDB Endowment*, vol. 13, no. 9, pp. 1401–1415, 2020.

[100] B.-C. Chien and S.-Y. He, "A hybrid approach for automatic schema matching," in *Proc. Int. Conf. Mach. Learn. Cybern*, 2010, pp. 2881–2886.

[101] F. Nargesian, E. Zhu, K. Q. Pu, and R. J. Miller, "Table union search on open data," in *Proc. VLDB Endowment*, vol. 11, no. 7, pp. 813–825, 2018.

[102] R. C. Fernandez et al., "Seeping semantics: Linking datasets using word embeddings for data discovery," in *Proc. IEEE Int. Conf. Data Eng.*, 2018, pp. 989–1000.

[103] T. Sahay, A. Mehta, and S. Jadon, "Schema matching using machine learning," in *Proc. 7th Int. Conf. Signal Process. Integr. Netw.*, 2020, pp. 359–366.

[104] C. Koutras, "Tabular schema matching for modern settings," 2024.

[105] Y. Zhang, M. Di, H. Luo, C. Xu, and R. T.-H. Tsai, "SMUTF: Schema matching using generative tags and hybrid features," 2024, *arXiv:2402.01685*.

[106] D. Qi and J. Wang, "CleanAgent: Automating data standardization with LLM-based agents," 2024, *arXiv:2403.08291*.

[107] M. M. H. Nahid and D. Rafiei, "NormTab: Improving symbolic reasoning in LLMs through tabular data normalization," 2024, *arXiv:2406.17961*.

[108] A. Sharma et al., "Automatic data transformation using large language model-an experimental study on building energy data," in *Proc. IEEE Int. Conf. Big Data*, 2023, pp. 1824–1834.

[109] M. J. Lee, J. Lin, and L.-T. Hsu, "Exploring the feasibility of automated data standardization using large language models for seamless positioning," in *Proc. 14th Int. Conf. Indoor Positioning Indoor Navigation*, 2024, pp. 1–6.

[110] A. Sharmaa et al., "DataMorpher: Automatic data transformation using LLM-based zero-shot code generation," in *Proc. IEEE Int. Conf. Data Eng.*, 2025, pp. 4536–4539.

[111] J. Xing et al., "Table-LLM-specialist: Language model specialists for tables using iterative generator-validator fine-tuning," 2024, *arXiv:2410.12164*.

[112] A. Brinkmann, N. Baumann, and C. Bizer, "Using LLMs for the extraction and normalization of product attribute values," in *Proc. Europ. Conf. Adv Databases Inf. Syst.*, 2024, pp. 217–230.

[113] V. Viswanathan, K. Gashteovski, K. Gashteovski, C. Lawrence, T. Wu, and G. Neubig, "Large language models enable few-shot clustering," *Trans. Assoc. Comput. Linguist.*, vol. 12, pp. 321–333, 2024.

[114] F. Thießen, J. D'Souza, and M. Stocker, "Probing large language models for scientific synonyms," in *Proc. 19th Int. Conf. Semantic Syst. Workshops*, 2023.

[115] Z. Jin, M. R. Anderson, M. Cafarella, and H. Jagadish, "Foofah: Transforming data by example," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2017, pp. 683–698.

[116] R. Singh, "Blinkfill: Semi-supervised programming by example for syntactic string transformations," in *Proc. VLDB Endowment*, vol. 9, no. 10, pp. 816–827, 2016.

[117] Z. Abedjan, J. Morcos, I. F. Ilyas, M. Ouzzani, P. Papotti, and M. Stonebraker, "DataXFormer: A robust transformation discovery system," in *Proc. IEEE Int. Conf. Data Eng.*, 2016, pp. 1134–1145.

[118] Y. He, X. Chu, K. Ganjam, Y. Zheng, V. Narasayya, and S. Chaudhuri, "Transform-data-by-example (TDE) an extensible search engine for data transformations," in *Proc. VLDB Endowment*, vol. 11, no. 10, pp. 1165–1177, 2018.

[119] C. Zuo, S. Assadi, and D. Deng, "Spine: Scaling up programming-by-negative-example for string filtering and transformation," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2022, pp. 521–530.

[120] Z. Jin, Y. He, and S. Chauduri, "Auto-transform: Learning-to-transform by patterns," in *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 2368–2381, 2020.

[121] D. Deng et al., "Unsupervised string transformation learning for entity consolidation," in *Proc. IEEE Int. Conf. Data Eng.*, 2019, pp. 196–207.

[122] J. Yang, Y. He, and S. Chaudhuri, "Auto-pipeline: Synthesizing complex data pipelines by-target using reinforcement learning and search," 2021, *arXiv:2106.13861*.

[123] W. Shen, Y. Yang, and Y. Liu, "Multi-view clustering for open knowledge base canonicalization," in *Proc. ACM Int. Conf. Knowl. Discov. Data Mining*, 2022, pp. 1578–1588.

[124] X. Lin and L. Chen, "Canonicalization of open knowledge bases with side information from the source text," in *Proc. IEEE Int. Conf. Data Eng.*, 2019, pp. 950–961.

[125] C. Jiang, Y. Jiang, W. Wu, Y. Zheng, P. Xie, and K. Tu, "Combo: A complete benchmark for open kg canonicalization," 2023, *arXiv:2302.03905*.

[126] M. Lomaeva and N. Jain, "Relation canonicalization in open knowledge graphs: A quantitative analysis," in *Proc. Eur. Semant. Web Conf.*, 2022, pp. 21–25.

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

[127] W. Shen, B. Yang, and Y. Liu, "Jointly canonicalizing and linking open knowledge base via unified embedding learning," in *Proc. ACM Web Conf.*, 2024, pp. 2304–2314.
[128] T.-H. Wu, B. Kao, Z. Wu, X. Feng, Q. Song, and C. Chen, "Mulce: Multi-level canonicalization with embeddings of open knowledge bases," in *Proc. 21st Int. Conf. Web Inf. Syst. Eng.*, 2020, pp. 315–327.
[129] W. Shen, Y. Li, Y. Liu, J. Han, J. Wang, and X. Yuan, "Entity linking meets deep learning: Techniques and solutions," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 3, pp. 2556–2578, Mar. 2023.
[130] W. Shen, J. Wang, and J. Han, "Entity linking with a knowledge base: Issues, techniques, and solutions," *IEEE Trans. Knowl. Data Eng.*, vol. 27, no. 2, pp. 443–460, Feb. 2015.
[131] Ö. Sevgili, A. Shelmanov, M. Arkhipov, A. Panchenko, and C. Biemann, "Neural entity linking: A survey of models based on deep learning," *Semantic Web*, vol. 13, no. 3, pp. 527–570, 2022.
[132] Y. Liu, W. Shen, Y. Wang, J. Wang, Z. Yang, and X. Yuan, "Joint open knowledge base canonicalization and linking," in *Proc. Int. Conf. Manag. Data*, 2021, pp. 2253–2261.
[133] J. S. Huh, C. Shin, and E. Choi, "Pool-search-demonstrate: Improving data-wrangling LLMs via better in-context examples," in *Proc. NeurIPS 2nd Table Rep. Learn. Workshop*, 2023.
[134] J. Ji, J. Kim, and Y. Kim, "Predicting missing values in survey data using prompt engineering for addressing item non-response," *Future Internet*, vol. 16, no. 10, 2024, Art. no. 351.
[135] X. He, Y. Ban, J. Zou, T. Wei, C. B. Cook, and J. He, "LLM-forest for health tabular data imputation," 2024, *arXiv:2410.21520*.
[136] M. Zhu, S. Stanivuk, A. Petrovic, M. Nikolic, and P. Lio, "Incorporating LLM priors into tabular learners," 2023, *arXiv:2311.11628*.
[137] A. Hayat and M. R. Hasan, "Claim your data: Enhancing imputation accuracy with contextual large language models," 2024, *arXiv:2405.17712*.
[138] L. Gondara and K. Wang, "MIDA: Multiple imputation using denoising autoencoders," in *Proc. Adv. Knowl. Discov. Data Min.: 22nd Pacific-Asia Conf.*, 2018, pp. 260–272.
[139] J. Yoon, J. Jordon, and M. Schaar, "Gain: Missing data imputation using generative adversarial nets," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 5689–5698.
[140] D. T. Neves, J. Alves, M. G. Naik, A. J. Proença, and F. Prasser, "From missing data imputation to data generation," *J. Comput. Sci.*, vol. 61, 2022, Art. no. 101640.
[141] A. Nazabal, P. M. Olmos, Z. Ghahramani, and I. Valera, "Handling incomplete heterogeneous data using vaes," *Pattern Recognit.*, vol. 107, 2020, Art. no. 107501.
[142] S. Zheng and N. Charoenphakdee, "Diffusion models for missing value imputation in tabular data," 2022, *arXiv:2210.17128*.
[143] S. Van Buuren and K. Groothuis-Oudshoorn, "mice: Multivariate imputation by chained equations in R," *J. Statist. softw.*, vol. 45, pp. 1–67, 2011.
[144] D. J. Stekhoven and P. Bühlmann, "Missforest—non-parametric missing value imputation for mixed-type data," *Bioinformatics*, vol. 28, no. 1, pp. 112–118, 2012.
[145] H.-F. Yu, N. Rao, and I. S. Dhillon, "Temporal regularized matrix factorization for high-dimensional time series prediction," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2016, pp. 847–855.
[146] T. Schnabel, A. Swaminathan, A. Singh, N. Chandak, and T. Joachims, "Recommendations as treatments: Debiasing learning and evaluation," in *Proc. Int. Conf. Mach. Learn.*, 2016, pp. 1670–1679.
[147] L. Li, L. Fang, and V. I. Torvik, "AutoDCWorkflow: LLM-based data cleaning workflow auto-generation and benchmark," 2024, *arXiv:2412.06724*.
[148] W. Ni, K. Zhang, X. Miao, X. Zhao, Y. Wu, and J. Yin, "IterClean: An iterative data cleaning framework with large language models," in *Proc. ACM Turing Award Celebration Conf.*, 2024, pp. 100–105.
[149] Z. A. Naeem, M. S. Ahmad, M. Eltabakh, M. Ouzzani, and N. Tang, "RetClean: Retrieval-based data cleaning using llms and data lakes," in *Proc. VLDB Endowment*, vol. 17, no. 12, pp. 4421–4424, 2024.
[150] S. Zhang, Z. Huang, and E. Wu, "Data cleaning using large language models," 2024, *arXiv:2410.15547*.
[151] A. Li et al., "Anomaly detection of tabular data using LLMs," 2024, *arXiv:2406.16308*.
[152] F. Biester, M. Abdelaal, and D. DelGaudio, "LLMClean: Context-aware tabular data cleaning via LLM-generated OFDS," in *Proc. Eur. Conf. Adv. Databases Inf. Syst.*, 2024, pp. 68–78.
[153] C. Northcutt, L. Jiang, and I. Chuang, "Confident learning: Estimating uncertainty in dataset labels," *J. Artif. Intell. Res.*, vol. 70, pp. 1373–1411, 2021.
[154] F. Neutatz, M. Mahdavi, and Z. Abedjan, "ED2: A case for active learning in error detection," in *Proc. 28th ACM Int. Conf. Inf. Knowl. Manage.*, 2019, pp. 2249–2252.
[155] A. Heidari, J. McGrath, I. F. Ilyas, and T. Rekatsinas, "Holodetect: Few-shot learning for error detection," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2019, pp. 829–846.
[156] A. Lew, M. Agrawal, D. Sontag, and V. Mansinghka, "PClean: Bayesian data cleaning at scale with domain-specific probabilistic programming," in *Proc. Int. Conf. Artif. Intell. Stat.*, 2021, pp. 1927–1935.
[157] M. Abdelaal, C. Hammacher, and H. Schoening, "Rein: A comprehensive benchmark framework for data cleaning methods in ML pipelines," 2023, *arXiv:2302.04702*.
[158] B. Karlaš et al., "Nearest neighbor classifiers over incomplete information: From certain answers to certain predictions," 2020, *arXiv:2005.05117*.
[159] M. Abdelaal, T. Ktitarev, D. Städtler, and H. Schöning, "SAGED: Few-shot meta learning for tabular data error detection," in *Proc. Int. Conf. Extending Database Technol.*, 2024, pp. 386–398.
[160] Z. Liu, Z. Zhou, and T. Rekatsinas, "Picket: Guarding against corrupted data in tabular data during learning and inference," *VLDB J.*, vol. 31, no. 5, pp. 927–955, 2022.
[161] J. Qin et al., "BClean: A Bayesian data cleaning system," in *Proc. IEEE Int. Conf. Data Eng.*, 2024, pp. 3407–3420.
[162] M. Mahdavi et al., "Raha: A configuration-free error detection system," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2019, pp. 865–882.
[163] C. Ge, Y. Gao, X. Miao, B. Yao, and H. Wang, "A hybrid data cleaning framework using Markov logic networks," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 5, pp. 2048–2062, 2020.
[164] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation-based anomaly detection," *ACM Trans. Knowl. Discov. Data*, vol. 6, no. 1, pp. 1–39, 2012.
[165] S. Giannakopoulou, M. Karpathiotakis, and A. Ailamaki, "Cleaning denial constraint violations through relaxation," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2020, pp. 805–815.
[166] X. Chu et al., "KATARA: A data cleaning system powered by knowledge bases and crowdsourcing," in *Proc. ACM SIGMOD Int. Conf. Manage. Data*, 2015, pp. 1247–1261.
[167] J. Liu, Y. Chabot, R. Troncy, V.-P. Huynh, T. Labbé, and P. Monnin, "From tabular data to knowledge graphs: A survey of semantic table interpretation tasks and methods," *J. Web Sem.*, vol. 76, 2023, pp. 100761.
[168] Q. Liu et al., "TAPEX: Table pre-training via learning a neural SQL executor," 2021, *arXiv:2107.07653*.
[169] Y. Ye, B. Hui, M. Yang, B. Li, F. Huang, and Y. Li, "Large language models are versatile decomposers: Decomposing evidence and questions for table-based reasoning," in *Proc. Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2023, pp. 174–184.
[170] Z. Wang et al., "Chain-of-table: Evolving tables in the reasoning chain for table understanding," 2024, *arXiv:2401.04398*.
[171] D. Ji et al., "Tree-of-table: Unleashing the power of LLMs for enhanced large-scale table understanding," 2024, *arXiv:2411.08516*.
[172] S. Huang, H. Li, Y. Gu, X. Hu, Q. Li, and G. Xu, "HyperG: Hypergraph-enhanced LLMs for structured knowledge," in *Proc. Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2025, pp. 1218–1228.
[173] K. Kong et al., "OpenTab: Advancing large language models as open-domain table reasoners," 2024, *arXiv:2402.14361*.
[174] Q. Mao, Q. Liu, Z. Li, M. Cheng, Z. Zhang, and R. Li, "Potable: Towards systematic thinking via stage-oriented plan-then-execute reasoning on tables," 2024, *arXiv:2412.04272*.
[175] D. Ritze and C. Bizer, "Matching web tables to DBpedia-A feature utility study," *Context*, vol. 42, no. 41, pp. 19–31, 2017.
[176] J. Qin and M. Iwaihara, "Annotating column type utilizing BERT and knowledge graph over wikipedia categories and lists," in *Proc. DEIM Forum*, 2022.
[177] M. Hulsebos, Ç. Demiralp, and P. Groth, "Gittables: A large-scale corpus of relational tables," in *Proc. ACM Manage. Data*, vol. 1, no. 1, pp. 1–17, 2023.
[178] X. Zhang et al., "TableLLM: Enabling tabular data manipulation by LLMs in real office usage scenarios," in *Proc. Findings Assoc. Comput. Linguist.*, 2025, pp. 10315–10344.
[179] Y. Sui, M. Zhou, M. Zhou, S. Han, and D. Zhang, "Table meets LLM: Can large language models understand structured table data? A benchmark and empirical study," in*Proc. ACM Int. Conf. Web Search Data Mining*, 2024, pp. 645–654.

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

[180] V. Cutrona, F. Bianchi, E. Jim énez-Ruiz, and M. Palmonari, "Tough tables: Carefully evaluating entity linking for tabular data," in *Proc. Int. Semantic Web Conf.*, Springer, 2020, pp. 328–343.

[181] A. Jiomekong and B. Foko, "Towards an approach based on knowledge graph refinement for tabular data to knowledge graph matching," in *Proc. Semantic Web Challenge Tabular Data Knowl. Graph Matching, Int. Semantic Web Conf.*, 2022, pp. 111–122.

[182] R. Avogadro, M. Cremaschi, E. Jiménez-Ruiz, and A. Rula, "A framework for quality assessment of semantic annotations of tabular data," in *Proc. Int. Semantic Web Conf.*, 2021, pp. 528–545.

[183] V.-P. Huynh, Y. Chabot, T. Labbé, J. Liu, and R. Troncy, "From heuristics to language models: A journey through the universe of semantic table interpretation with DAGOBAH," in *Proc. 21st Int. Semantic Web Conf.*, Vol. 3320, 2022.

[184] N. Abdelmageed, S. Schindler, and B. König-Ries, "BiodivTab: A table annotation benchmark based on biodiversity research data," in *Proc. Semantic Web Challenge Tabular Data Knowl. Graph Matching, Int. Semantic Web Conf.*, 2021, pp. 13–18.

[185] F. Belotti, F. Dadda, M. Cremaschi, R. Avogadro, and M. Palmonari, "Evaluating LLMs on entity disambiguation in tables," 2024, *arXiv:2408.06423*.

[186] F. Belotti, M. Cremaschi, F. Dadda, R. Avogadro, and M. Palmonari, "How good are LLMs in disambiguating entities in tabular data? A comprehensive study," *Data Knowl. Eng.*, vol. 164, 2026, Art. no. 102596.

[187] S. Chen et al., "Linkingpark: An automatic semantic table interpretation system," *J. Web Semantics*, vol. 74, 2022, Art. no. 100733.

[188] K. Korini, R. Peeters, and C. Bizer, "SOTAB: The WDC schema. org table annotation benchmark," in *Proc. CEUR Workshop Proc.*, 2022, pp. 14–19.

[189] E. Jiménez-Ruiz, O. Hassanzadeh, V. Efthymiou, J. Chen, and K. Srinivas, "Semtab2019: Resources to benchmark tabular data to knowledge graph matching systems," in *Proc. Semantic Web: 17th Int. Conf.*, 2020, pp. 514–530.

[190] P. Nguyen, I. Yamada, N. Kertkeidkachorn, R. Ichise, and H. Takeda, "MTab4wikidata at semtab 2020: Tabular data annotation with wikidata," in *Proc. Semantic Web Challenge Tabular Data Knowl. Graph Matching, Int. Semantic Web Conf.*, 2020, pp. 86–95.

[191] C. S. Bhagavatula, T. Noraset, and D. Downey, "Tabel: Entity linking in web tables," in *Proc. Int. Semantic Web Conf.*, Springer, 2015, pp. 425–441.

[192] Z. Wang et al., "TUTA: Tree-based transformers for generally structured table pre-training," in *Proc. ACM Int. Conf. Knowl. Discov. Data Mining*, 2021, pp. 1780–1790.

[193] Y. Dong, C. Xiao, T. Nozawa, M. Enomoto, and M. Oyamada, "Deepjoin: Joinable table discovery with pre-trained language models," 2022, *arXiv:2212.07588*.

[194] S. Singh, A. F. Aji, G. S. Tomar, and C. Christodoulopoulos, "A relation extraction dataset for knowledge extraction from web tables," in *Proc. Int. Conf. Comput. Linguist.*, 2022, pp. 2319–2327.

[195] M. Cremaschi, B. Spahiu, M. Palmonari, and E. Jimenez-Ruiz, "Survey on semantic interpretation of tabular data: Challenges and directions," 2024, *arXiv:2411.11891*.

[196] W. Ni, X. Miao, X. Zhao, Y. Wu, and J. Yin, "Automatic data repair: Are we ready to deploy," 2023, *arXiv:2310.00711*.

[197] T. Bendinelli, A. Dox, and C. Holz, "Exploring LLM agents for cleaning tabular machine learning datasets," 2025, *arXiv:2503.06664*.

[198] X. Zhou et al., "A survey of LLM data," 2025, *arXiv:2505.18458*.

[199] M. Cremaschi, F. Belotti, J. D'Souza, and M. Palmonari, "Mammotab 25: A large-scale dataset for semantic table interpretation-training, testing, and detecting weaknesses," in *Proc. Int. Semantic Web Conf.*, Springer, 2025, pp. 131–148.

[200] M. Cremaschi, F. D'Adda, and A. Maurino, "stEELLM: An LLM for generating semantic annotations of tabular data," *ACM Trans. Intell. Syst. Technol.*, 2025.

[201] M. Cremaschi et al., "s-elBat: A semantic interpretation approach for messy table-s," in *Proc. Semantic Web Challenge Tabular Data Knowl. Graph Matching, Int. Semantic Web Conf.*, 2022, pp. 59–71.

[202] J. Qiu et al., "Dependency-aware core column discovery for table understanding," in *Proc. Int. Semantic Web Conf.*, 2023, pp. 159–178.

[203] R. Peeters, R. C. Der, and C. Bizer, "WDC products: A multi-dimensional entity matching benchmark," 2023, *arXiv:2301.09521*.

[204] R. Peeters and C. Bizer, "Using ChatGPT for entity matching," in *Proc. Eur. Conf. Adv. Databases Inf. Syst.*, Springer, 2023, pp. 221–230.

[205] R. Aly et al., "FEVEROUS: Fact extraction and VERification over unstructured and structured information," in *Proc. 35th Conf. Neural Inf. Process. Syst. Datasets Benchmarks Track*, 2021, pp. 632–642).

[206] R. Aly et al., "The fact extraction and verification over unstructured and structured information (feverous) shared task," in *Proc. 4th Workshop Fact Extraction VERification*, 2021, pp. 1–13.

[207] M. Saeed, G. Alfarano, K. Nguyen, D. Pham, R. Troncy, and P. Papotti, "Neural re-rankers for evidence retrieval in the feverous task," in *Proc. 4th Workshop Fact Extraction VERification*, 2021, pp. 108–112.

[208] Z. Wu, N. Hu, and Y. Feng, "Enhancing structured evidence extraction for fact verification," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2023, pp. 6631–6641.

[209] Y. Lou, B. Kuehl, E. Bransom, S. Feldman, A. Naik, and D. Downey, "S2abEL: A dataset for entity linking from scientific tables," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2023, pp. 3089–3101.

[210] I. Jayawardene, R. Avogadro, A. Soylu, and D. Roman, "TablinkLLM: An LLM-based approach for entity linking in tabular data," in *Proc. IEEE/WIC Int. Conf. Web Intell. Intell. Agent Technol.*, 2024, pp. 206–214.

[211] M. Färber, D. Lamprecht, J. Krause, L. Aung, and P. Haase, "SemOpenAlex: The scientific landscape in 26 billion RDF triples," in *Proc. Int. Semantic Web Conf.*, Springer, 2023, pp. 94–112.

[212] P. O'Neil, E. O'Neil, X. Chen, and S. Revilak, "The star schema benchmark," University of Massachusetts, Boston, Tech. Rep., 2007.

[213] K. P. Gaffney, M. Prammer, L. Brasfield, D. R. Hipp, D. Kennedy, and J. M. Patel, "SQLite: Past, present, and future," in *Proc. VLDB Endowment*, vol. 15, no. 12, pp. 3535–3547, 2022.

[214] Y. Yuan, M. F. Salmi, Y. Huai, K. Wang, R. Lee, and X. Zhang, "Spark-GPU: An accelerated in-memory data processing engine on clusters," in *Proc. IEEE Int. Conf. Big Data*, 2016, pp. 273–283.

[215] J. Ortiz, B. Lee, M. Balazinska, J. Gehrke, and J. L. Hellerstein, "{SLAOrchestrator }: Reducing the cost of performance { SLAs} for cloud data analytics," in *Proc. USENIX Annu. Techn. Conf.*, 2018, pp. 547–560.

[216] P. S. G. C. C. G. P. K. Y. G. Sanjib Das, A. Doan, and D. Paulsen, "The magellan data repository," 2015. [Online]. Available: https://sites.google.com/site/anhaidgroup/usefulstuff/data

[217] S. Rayana, "Odds library," 2016.

[218] N. F. Liu et al., "Lost in the middle: How language models use long contexts," in *Proc. Conf. Assoc. Comput. Linguist.*, 2024, pp. 157–173.

[219] X. Sui, Y. Zhang, K. Song, B. Zhou, and X. Yuan, "Multi-level feature interaction for open knowledge base canonicalization," *Knowl.-Based Syst.*, vol. 303, 2024, Art. no. 112386.

[220] A. Singha, J. Cambronero, S. Gulwani, V. Le, and C. Parnin, "Tabular representation, noisy operators, and impacts on table structure understanding tasks in LLMs," 2023, *arXiv:2310.10358*.

[221] T. Liu, F. Wang, and M. Chen, "Rethinking tabular data understanding with large language models," 2023, *arXiv:2312.16702*.

[222] W. Hwang, J. Yim, S. Park, and M. Seo, "A comprehensive exploration on WikiSQL with table-aware word contextualization," 2019, *arXiv:1902.01069*.

[223] S. Jaitly, T. Shah, A. Shugani, and R. S. Grewal, "Towards better serialization of tabular data for few-shot classification," 2023, *arXiv:2312.12464*.

[224] S. Hegselmann, A. Buendia, H. Lang, M. Agrawal, X. Jiang, and D. Sontag, "TabLLM: Few-shot classification of tabular data with large language models," in *Proc. Int. Conf. Artif. Intell. Statist.*, 2023, pp. 5549–5581.

[225] Z. Wang, C. Gao, C. Xiao, and J. Sun, "MediTab: Scaling medical tabular data predictors via data consolidation, enrichment, and refinement," 2023, *arXiv:2305.12081*.

[226] H. Gong et al., "TableGPT: Few-shot table-to-text generation with table structure reconstruction and content matching," in *Proc. Int. Conf. Comput. Linguist.*, 2020, pp. 1978–1988.

[227] B. Zhao et al., "Large language models are complex table parsers," 2023, *arXiv:2312.11521*.

[228] Q. Dong et al., "A survey on in-context learning," 2022, *arXiv:2301.00234*.

[229] Y. Lu, M. Bartolo, A. Moore, S. Riedel, and P. Stenetorp, "Fantastically ordered prompts and where to find them: Overcoming few-shot prompt order sensitivity," 2021, *arXiv:2104.08786*.

[230] J. Liu, D. Shen, Y. Zhang, B. Dolan, L. Carin, and W. Chen, "What makes good in-context examples for GPT-3?," 2021, *arXiv:2101.06804*.

[231] X. Wen, H. Zhang, S. Zheng, W. Xu, and J. Bian, "From supervised to generative: A novel paradigm for tabular deep learning with large language models," in *Proc. ACM Int. Conf. Knowl. Discov. Data Mining*, 2024, pp. 3323–3333.

[232] A. Su et al., "TableGPT2: A large multimodal model with tabular data integration," 2024, *arXiv:2411.02059*.

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.

[233] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Representations*, vol. 1, no. 2, p. 3, 2022.
[234] Z. Ding, J. Tian, Z. Wang, J. Zhao, and S. Li, "Data imputation using large language model to accelerate recommender system."
[235] H. Liu et al., "Few-shot parameter-efficient fine-tuning is better and cheaper than in-context learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 1950–1965.
[236] J. Zhang et al., "CoddLLM: Empowering large language models for data analytics," 2025, *arXiv:2502.00329*.
[237] M. Mahdavi and Z. Abedjan, "Baran: Effective error correction via a unified context representation and transfer learning," in *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 1948–1961, 2020.
[238] W. McKinney and P. Team, "Pandas-powerful python data analysis toolkit," *Pandas—Powerful Python Data Anal. Toolkit*, vol. 1625, 2015.
[239] W.-H. Chen, W. Tong, A. Case, and T. Zhang, "Dango: A mixed-initiative data wrangling system using large language model," in *Proc. CHI Conf. Hum. Factors Comput. Syst.*, 2025, pp. 1–28.
[240] M. Zhang, O. Press, W. Merrill, A. Liu, and N. A. Smith, "How language model hallucinations can snowball," 2023, *arXiv:2305.13534*.
[241] Y. Luo, Z. Yang, F. Meng, Y. Li, J. Zhou, and Y. Zhang, "An empirical study of catastrophic forgetting in large language models during continual fine-tuning," 2023, *arXiv:2308.08747*.
[242] N. Gruver, M. Finzi, S. Qiu, and A. G. Wilson, "Large language models are zero-shot time series forecasters," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2024, pp. 19622–19635.
[243] M. Zheng et al., "Multimodal table understanding," 2024, *arXiv:2406.08100*.



Ying He received the MS degree from Soochow University, China. She is currently working toward the PhD degree in computer science and technology with Fudan University, China. Her research interests include data governance, controlled text generation, and large language model (LLM).



Qiang Yang received the PhD degree from the King Abdullah University of Science and Technology, Saudi Arabia. He is currently working as a postdoc associate with the Medical of School, University of Florida, US. His research interests include interpretable graph learning, bioinformatics, biomedicine, and protein language models.



Yuxiao Yang received the bachelor's degree in intelligent science and technology from Northwest University, China. He is currently working toward the master's degree with the Renmin University of China. His research interests include large language models (LLMs), particularly LLM-based agents and their applications in report generation and data cleaning.



Yubo Zhou received the graduate degree from Zhejiang University. He is currently working toward the master's degree in computer science and technology with Fudan University in China. His research interests include data governance and large language models.



Zhecheng Hu is currently working toward the MS degree in computer science and technology with Fudan University, China. His research interests include large language models (LLMs) and data wrangling.



Sheng Yuan is currently working toward the master's degree in electronic information with the Renmin University of China. His current main research directions include large language models, data governance, and natural language processing.



Yanghua Xiao received the PhD degree in software theory from Fudan University, Shanghai, China, in 2009. He is a professor of computer science with Fudan University. He is the director of Knowledge Works Lab, Fudan University. He is one of young 973 scientists. His research interest includes Big Data management and mining, graph database, knowledge graph. He was a visiting professor with Human Genome Sequencing Center, Baylor College Medicine, and visiting researcher with Microsoft Research Asia.



Zhixu Li received the PhD degree in computer science from the University of Queensland, in 2013. He is a professor with the School of Information & School of Smart Governance, Renmin University of China. He used to work as a professor with Fudan University (2021-2024) and Soochow University (2014-2021). His main research interests include, but not limited to data governance, knowledge graph and large language models.

Authorized licensed use limited to: University of Arkansas Litte Rock. Downloaded on November 09,2026 at 22:01:57 UTC from IEEE Xplore. Restrictions apply.